

Feb

2025

Crack Interview

02	03	04	05	06	07	08
					8 PM FS2	2 PM FS2
09	10	11	12	13	14	15
9:30 PM FS1	9:30 PM FS1		8:00 PM FS3	12 PM FS3	8 PM FS6	9:30 PM FS4
16	17	18	19	20	21	22
9:30 PM FS4		9:30 PM FS5		Ownership 9:30 PM	2 PM - OS 9 PM - OS	
23	24	25	26	27	28	
						

Set-1

04-02-2025

Netflix

Product Sense

Assume you are working at a major video streaming service, where people are able to stream different kinds of contents on the platform.

This content can be like Movies or TV shows etc.

This also means that you source content from various studios and stream on your platform. The product was launched a few months ago and we would like to assess the health and performance of this product.

Clarifying questions

when we meant health and performance or are we focusing on the use of satisfaction mainly or is it OK if you also focus on the matrix like retention and shown as well?

And oh, did we launch this globally all at once or did we roll out this region by region? Are there any specific markets or regions that we want to focus the sentiment analysis?

What kind of areas would you look at to understand the overall health and performance of the product?

- Viewership
- Engagement
- User Satisfaction
- Retention
- Revenue
- Content Recommendation
- Customer Acquisition.

.What metrics would you look at to understand the performance in all of the above (Engagement) areas?

- DAU
- MAU ,stickiness ratio (DAU,MAU)
- Total Views
- Unique users
- Avg session user
- session_interval
- session_duration
- total_no_of_sessions
- total_watched_duration
- Completion rate
- Engagement rate
- Subscription conversion rate
- Avg watch time
- Click through rate
- Retention rate
- churn rate
- sentiment score
- csat score
- total revenue
- avg revnue per user
- Subscription revenue
- Content Discovery rate

Based on the image provided, here are the **definitions and formulas** for the engagement metrics listed:

User Acquisition & Traffic Metrics

- **DAU (Daily Active Users)**: Count of unique users who engage with the platform in a single day
 - **Formula**: Count of distinct users per day.
- **MAU (Monthly Active Users)**: Count of unique users who engage with the platform in a 30-day period
 - **Formula**: Count of distinct users per month
- **Total Views**: Total number of page views or content views
 - **Formula**: Sum of all views
- **Unique Users**: Count of distinct individual users
 - **Formula**: Count of unique user identifiers

Session Metrics

- **Avg Session User**: Average number of sessions per user
 - **Formula**: $\text{Total Sessions} \div \text{Total Users}$.
- **Session Interval**: Time between consecutive sessions
 - **Formula**: $\text{Time of Current Session Start} - \text{Time of Previous Session End}$
- **Session Duration**: Length of time a user spends in a single session
 - **Formula**: $\text{Session End Time} - \text{Session Start Time}$
- **Total No of Sessions**: Total count of all user sessions
 - **Formula**: Sum of all sessions
- **Total Watched Duration**: Cumulative time users spend watching content
 - **Formula**: Sum of all watch times

Engagement Rate Metrics

- **Completion Rate**: Percentage of users who complete an action/content
 - **Formula**: $(\text{Completions} \div \text{Total Starts}) \times 100$
- **Engagement Rate**: Percentage of users who interact with content
 - **Formula**: $(\text{Engaged Users} \div \text{Total Users}) \times 10$
- **Avg Watch Time**: Average duration users spend watching content
 - **Formula**: $\text{Total Watch Time} \div \text{Number of Views}$
- **Click Through Rate (CTR)**: Percentage of users who click on a link/CTA
 - **Formula**: $(\text{Total Clicks} \div \text{Total Impressions}) \times 100$

Retention & Loyalty Metrics

- **Retention Rate**: Percentage of users who return over a period
 - **Formula**: $(\text{Users at End of Period} \div \text{Users at Start of Period}) \times 100$
- **Churn Rate**: Percentage of users who stop using the service
 - **Formula**: $(\text{Users Lost} \div \text{Total Users at Start}) \times 100$

Sentiment & Satisfaction Metrics

- **Sentiment Score**: Measure of user sentiment (positive/negative)
 - **Formula**: $(\text{Positive Mentions} - \text{Negative Mentions}) \div \text{Total Mentions}$
- **CSAT Score (Customer Satisfaction)**: Percentage of satisfied customers
 - **Formula**: $(\text{Satisfied Customers} \div \text{Total Respondents}) \times 100$

Revenue Metrics

- **Total Revenue**: Sum of all revenue generated
 - **Formula**: Sum of all transactions
- **Avg Revenue Per User (ARPU)**: Average revenue generated per user
 - **Formula**: $\text{Total Revenue} \div \text{Total Users}$
- **Subscription Revenue**: Revenue from subscription-based services
 - **Formula**: Sum of all subscription payments

Discovery Metrics

- **Content Discovery Rate**: Percentage of users who discover new content
 - **Formula**: $(\text{Users Who Discovered New Content} \div \text{Total Users}) \times 100$

[The text has been generated with the help of AI]

.What are the most important dimensions/ cuts that would be relevant?

User Demographics

- Age
 - Gender
 - Signup date
 - Location
 - Language
- Device
 - Make
 - Model
 - version

Content

- Type
 - Genre
 - Duration
 - Language
 - Categories
- Location
 - Region
 - City
 - Location
 - Zip code

User Behaviour

- New vs Returning user
- Viewing pattern (Total views, avg session interval ...)
- Engagement Patterns (upvotes, . t downvotes)

,What kind of visualizations would you give depending on the metrics and cuts?

DAU/MAU over time --- line chart

x-axis = time

yaxis = DAU/MAU

line chart segmented by age group

- avg view time for content types ---- grouped bar chart

x-axis = time

y axis = avg ciew duration --- segmented by bars by content type

- revneu generated by ads by channel type
pie chart

- Funnel chart (acquisition funnel)

Visitors -> sign ups -> free trials -> paid subscribers

Data Modeling

,Considering the metrics discussed above, can you design a data model for the streaming service? What are the tables and their relationships?

Users watch content generated by studios in the platform (tv show, movie)
users can upvote, downvote recommended

Key Entities

- User
- Content
- Device
- Location
- Events

dim_location

- location_id (PK)
- Region
- country
- zip code
- date (snapshot partitioned)

Data Model

dim_user

- user_id (PK)
- age
- Gender
- signup_date
- location
- language
- date (snapshot partitioned)

dim_content

- content_id (PK)
-
- type
- Genre
- category
- language
- creator_id
- date (snapshot partitioned)

dim_device

- device_id (PK)
- Make
- model
- version
- date (snapshot partitioned)

fct_events

- event_id (PK)
- user_id
- content_id
- device_id,
- session_id
- location_id
- event_type (like, watch, react)
- event_value
- event_start_ts
- event_end_ts
- date (incrementally partitioned)

relationships -

dim -> fact (1:M)
fact -> dim (1:1)

For each and every episode customer is able to leave a rating. How can I track that model?

grouping sets allows to specify
multiple grouping combinations
at once

Instead of writing multiple
group by queries &
combining them with union all

Considering the data model and metrics you have proposed, we want to build aggregated datasets/tables at multiple different grains for analytics purposes. Provide high level schema for some of the datasets/tables

`agg_viewership_stats`

- `show_name`
- `content_type`
- `daily_active_users`
- `monthly_active_users`
- `completion_rate`
- `engagement_rate`
- `date ( partitioned)`

`(grouping sets)`

generate multiple levels
of aggregation

SQL

Table 1: fct_view_logs

```
-- Schema
column_name      | data_type
-----|-----
session_id       | varchar
user_id          | bigint
content_id       | bigint
start_ts         | timestamp
end_ts           | timestamp
date             | varchar
```

SAMPLE DATA (fct_view_logs)

session_id	user_id	content_id	start_ts	end_ts	date (PARTITION)
abc123	1001	1	09:08:00	10:17:00	2024-01-01
def456	1002	2	10:14:00	10:22:00	2024-01-01
jk1012	1003	4	11:46:00	13:59:00	2024-01-01
nj1234	1004	3	13:10:00	NULL	2024-01-01

Table 2: dim_content

```
-- Schema
column_name      | data_type
-----|-----
content_id       | bigint
content_type     | varchar (values: series/movie/documentary)
title           | varchar
genre           | varchar
studio_id       | varchar
creation_date    | date
date            | date
```

SAMPLE DATA (dim_content)

content_id	content_type	title	genre	studio_id	creation_date
1	series	The Office	Comedy	NBC	2005-03-24
2	movie	Interstellar	Sci Fiction	ECP	2014-10-26
3	documentary	Planet Earth	Documentary	BBC	2006-03-05
4	series	Breaking Bad	Crime Drama	EPP	2017-10-29
5	documentary	Blue Planet	Documentary	AMC	1972-03-15

Q1: For content that has been watched more than 10 minutes,
list all the content_ids along with num_viewers and total_view_time.

```
SELECT
    content_id,
    COUNT(DISTINCT user_id) AS num_viewers,
    SUM(EXTRACT(EPOCH FROM (end_ts - start_ts))) AS
total_view_time
FROM fct_view_logs
WHERE event_type = 'view'
HAVING SUM(EXTRACT(EPOCH FROM (end_ts - start_ts))) > 10
```

Q2: Output Table: cumulative_studio_metrics. This table stores multiple metrics for each studio.

Output Table: cumulative_studio_metrics

column_name	data_type
date	date
studio_id	varchar
total_view_time_1d	bigint
total_view_time_lifetime	bigint

Requirement:

We want to update the table with new metrics data every day.

Can you write a SQL query to accomplish this using the tables provided above?

```
WITH studio_stats AS (
    SELECT
        studio_id,
        date,
        content_id
    FROM dim_content
    WHERE date = current_date()
)

events_stats AS (
    SELECT
        content_id,
        EXTRACT (EPOCH FROM (end_ts - start_ts)) AS view_time
    FROM fct_view_logs
    WHERE date = current_date()
),

today_metrics AS (
    SELECT
        ss.date,
        ss.studio_id,
        SUM(es.view_time) AS total_view_time_1d
    FROM events_stats AS es
    LEFT JOIN studio_stats AS ss ON es.content_id = ss.content_id
    GROUP BY date, studio_id
),

cumulative_studio_stats AS (
    SELECT *
    FROM cumulative_studio_metrics
    WHERE date = current_date() - 1
)

SELECT
    tm.date,
    COALESCE(tm.studio_id, csm.studio_id) AS studio_id,
    COALESCE(tm.total_view_time_1d, 0) AS total_view_time_1d,
    COALESCE(tm.total_view_time_lifetime, 0) + COALESCE(csm.total_view_time_lifetime, 0) AS total_view_time_lifetime
FROM today_metrics AS tm
FULL OUTER JOIN cumulative_studio_metrics AS csm ON tm.studio_id = csm.studio_id
```

Practice SQL

Fact Table: fact_event_logs (today)

- session_id
- user_id
- content_id
- event_type (view, like, comment, share etc.)
- start_timestamp
- end_timestamp
- device_id
- location_id
- date (partition)

Dimension Table: dim_content (today)

- content_id
- channel_id
- content_type
- title
- genre
- creation_date
- content_length
- language
- date (partition)

Dimension Table: dim_channel (yesterday)

- channel_id
- viewers_1d = number of viewers in 1 day
- Sessions_1d = number of sessions in 1 day
- Viewers_ltv = num_viewers lifetime value, is it cumulative number (adding up everyday?)
- sessions_ltv = num_sessions lifetime value, is it cumulative number (adding up everyday?)
- date (partition)

.For all YouTube videos that were watched for more than 10 minutes today, list all video_ids along with the number of viewers and total watch time (in minutes)

```
select
    content_id,
    count(distinct user_id) as num_viewers,
    sum(extract(epoch from (end_timestamp - start_timestamp))) / 60 as
total_watchtime_mins
from fact_event_logs
where event_type = 'view' and date = current_date
group by content_id
having sum(extract(epoch from (end_timestamp - start_timestamp))) / 60 > 10
```

left join is used so all content generated today can be viewed even if there 0 views. Full outer join is useful to include all channels even if there is activity or not

We have data in the dim_channel table until yesterday, using the latest date partition data from fact_event_logs and dim_content tables. Can we prepare the latest date partition data for a dim_channel table that stores aggregate metrics by YouTube channel?

```
with today_metrics as (  
    select  
        dim.channel_id,  
        count(distinct fct.user_id) as viewers_1d,  
        count(distinct fct.session_id) as sessions_1d  
    from (select content_id, channel_id from dim_content where date =  
current_date) dim  
    left join (select content_id, user_id, session_id from  
fact_event_logs where event_type = 'view' and date = current_date) fct  
        on fct.content_id = dim.content_id  
    group by dim.channel_id  
,  
  
yesterday_metrics as (  
    select *  
    from dim_channel  
    where date = current_date - 1  
)  
  
select  
    coalesce(tm.channel_id, ym.channel_id) as channel_id,  
    coalesce(tm.viewers_1d, 0) as viewers_1d,  
    coalesce(tm.sessions_1d, 0) as sessions_1d,  
    coalesce(tm.viewers_1d, 0) + coalesce(ym.viewers_ltv, 0) as  
Viewers_ltv,  
    coalesce(tm.sessions_1d, 0) + coalesce(ym.sessions_ltv, 0) as  
sessions_ltv  
from today_metrics tm  
full outer join yesterday_metrics ym  
    on tm.channel_id = ym.channel_id
```

PYTHON

Given a stream of user ratings, calculate the average rating for each content.

```
-----

# input stream:
# user_id, content_name, genre, rating

stream = [
    (1001, "The Office",      "Comedy",    3.0),
    (1002, "The Godfather",   "Drama",    4.0),
    (1003, "The Notebook",    "Romance",  5.0),
    (1043, "Her",             "Sci-Fi",   3.0),
    (1004, "Interstellar",     "Sci-Fi",   2.0),
    (1011, "Breaking Bad",     "Drama",    4.0),
    (1032, "Interstellar",     "Sci-Fi",   3.0),
    (1124, "Big Bang Theory",  "Comedy",   1.0),
    (1011, "The Godfather",    "Drama",    3.0),
    (1052, "Interstellar",     "Sci-Fi",   3.0),
    (1226, "Big Bang Theory",  "Comedy",   5.0),
    (2164, "The Martian",      "Sci-Fi",   4.0),
    (2451, "Back To The Future", "Sci-Fi",   4.0)
]

# Expected Output
content_ratings = {
    "The Office": 3.0,
    "The Godfather": 3.5,
    "The Notebook": 5.0,
    "Her": 3.0,
    "Interstellar": 2.666..., # (example of average across multiple ratings)
    "Breaking Bad": 4.0,
    "Big Bang Theory": 3.0,
    "The Martian": 4.0,
    "Back To The Future": 4.0
}
```

Additional:

Write a function that returns the **top 3 titles per theme** based on the **highest average rating**. If a theme has fewer than 3 titles, return all of them sorted by average rating descending.

Input dictionaries

```
content_ratings = {
    "The Office": 3.0,
    "The Godfather": 3.5,
    "The Notebook": 3.0,
    "Interstellar": 3.3,
    "Breaking Bad": 3.4,
    "Big Bang Theory": 3.0,
    "The Martian": 3.2,
    "Back To The Future": 3.1
}
```

```
content_genre = {
    "The Office": "Comedy",
    "The Godfather": "Drama",
    "The Notebook": "Romance",
    "Her": "Sci-Fi",
    "Interstellar": "Sci-Fi",
    "Breaking Bad": "Drama",
    "Big Bang Theory": "Comedy",
    "The Martian": "Sci-Fi",
    "Back To The Future": "Sci-Fi"
}
```

...

```
Top 2 videos in genre 'GenreName':
- Video Title | Genre: GenreName | Rating: X.X
- Video Title | Genre: GenreName | Rating: X.X
```

...

```
from collections import defaultdict
```

```
def top_3_per_genre(content_ratings, content_genre):
```


Set-2

Feb 8th 2025

Facebook NewsFeed

Product Sense

A feed is a stream of information that appears on a website or app. It can include posts, stories, articles, and other types of content. The feed is usually personalized to show you the most relevant and interesting information based on your past interactions with the site or app. One such example of a feed is the Facebook Newsfeed where content you may be suggested could include:

- => Text or image posts from Friends & Family
- => Posts from groups you may be in
- => People you can add as friends
- => Pages/Businesses you might want to follow

Let's say we recently launched a new type of content (e.g., uploading short videos) in the last couple of months. The Product Manager (PM) of the team would like to know how this content is performing and if it is adding value.

clarifying questions

- > Is the short video launched globally or only in specific regions ?
- > Does the short video content appear in the main feed, or does it have a dedicated Session ?

Question 1 :

What business questions would help us understand how newsfeed is performing after launching new content? (8-10)

Engagement

- > Are users interacting with the short videos?
- > Has overall engagement rate impacted by short videos?

Viewership

- > How is viewership on short videos compare to other content?
- > And users spending more time on the site after launch?

Retention

- > Has retention improved since introducing short videos?
- > How has churn been affected by new feature?

Revenue

- > Has ad revenue shown any noticeable change post launch?
- > Is the Avg revenue per user improved?

User satisfaction

- > Has users satisfied with the new content?
- > Have we seen decline or rise in the new users ?

Question 2 :
What metrics would you define to help answer the business questions from Q1? (8 - 10)

- DAU (Daily Active Users)
- MAU (Monthly Active Users)
- Enagagement Rate
- Total views
- Unique users
- Avg Time Spent
- Sentiment Score
- NPS Score
- CSAT Score
- Rentention Rate
- Churn Rate
- Ad Revenue
- avg revenue per user

1000 users

$$\frac{800}{1000} \times 100 = 80\%$$
$$\frac{200}{1000} \times 100 = 20\%$$

(Based on the image showing a list of metrics, here are the definitions and formulas (numerator / denominator) for key metrics:

- DAU (Daily Active Users)**
 - **Definition**: Number of unique users who engage with your product / service in a single day
 - **Formula**: Count of unique users active on a given day
- MAU (Monthly Active Users)**
 - **Definition**: Number of unique users who engage with your product / service within a 30-day period
 - **Formula**: Count of unique users active in a given month
- Engagement Rate**
 - **Definition**: Percentage of users who actively interact with content
 - **Formula**: (Number of engaged users / Total users) × 100
- Retention Rate**
 - **Definition**: Percentage of users who continue using the product over time
 - **Formula**: (Users at end of period / Users at start of period) × 100
- Churn Rate**
 - **Definition**: Percentage of users who stop using the product
 - **Formula**: (Users lost during period / Users at start of period) × 100
- NPS Score (Net Promoter Score)**
 - **Definition**: Customer loyalty metric
 - **Formula**: % Promoters (9-10 ratings) - % Detractors (0-6 ratings)
- CSAT Score (Customer Satisfaction)**
 - **Definition**: Measure of customer satisfaction
 - **Formula**: (Number of satisfied customers / Total survey responses) × 100
- Average Revenue Per User (ARPU)**
 - **Definition**: Revenue generated per user
 - **Formula**: Total Revenue / Total Number of Users

[The text has been generated with the help of AI]

Question 3 :

What are the most important dimensions/cuts that would be relevant for your metrics?

Content

- type
- duration
- quality
- language
- origin_country
- created_date
- genre

user

- type
- interests
- age
- gender
- signup_date
- pref_language
- origin_country

User behavior

- new vs returning
- frequency of use
- viewing patterns

Time

- day of week
- weekend vs weekday
- seasonality

Device

- make
- model
- app version
- browser type

Location:

- country
- region
- city
- zip

Question 4:

How would you visualize your metrics and dimensions in a dashboard? (2 C, 1 S)

DAU trend over time, breakdown by content type => multi line chart

X: time

Y: DAU

Total Views trend over time, breakdown by Genre => stacked bar chart

X: ~~time~~ genres

Y: Total Views

Revenue sliced by content type => Pie Chart

Question 5 (If >2 mins in this section remaining):

What kind of automatic checks or monitoring would you add to your metrics to detect potential issues?

- Data Quality checks
- freshness checks
- data consistent
- data integrity checks
- Anomaly Detection
- Statistical Analysis
- User Feedback checks (NPS, CSAT)

Based on the image, here are brief explanations for each automatic check/monitoring type:

Data Quality Checks

- Validates that data meets predefined quality standards, checking for completeness, accuracy, and validity of data fields

Freshness Checks

- Monitors whether data is being updated within expected timeframes, alerting when data becomes stale or outdated

Data Consistent

- Ensures data remains uniform across different systems, tables, or time periods, checking for contradictions or mismatches

Data Integrity Checks

- Verifies that data maintains its accuracy and consistency throughout its lifecycle, including referential integrity and constraint validation

Anomaly Detection

- Identifies unusual patterns or outliers in data that deviate from expected behavior

Statistical Analysis (Detailed explanation using SD)

- Uses **standard deviation (SD)** to detect anomalies by measuring how far data points deviate from the mean
- Typically, values beyond **2-3 standard deviations** from the mean are flagged as anomalies
- **Example**: If average daily transactions = 1000 with SD = 100, values below 700 or above 1300 (3 SD away) would trigger alerts
- Can use **z-scores** to standardize measurements: $z = (\text{value} - \text{mean}) / \text{SD}$
- Helps identify sudden spikes, drops, or unusual trends in metrics automatically

User Feedback Checks

- Monitors user satisfaction metrics and feedback to detect potential issues

NPS, CSAT

- **NPS (Net Promoter Score)**: Measures customer loyalty and likelihood to recommend
- **CSAT (Customer Satisfaction)**: Tracks satisfaction levels with specific interactions or products

$$\sigma = \sqrt{\sum (x - \mu)^2 / N}$$

μ = mean
 N = total values
 x = each value

Data Modeling

We would like to design a data model to be able to answer our business questions

With this data model, we wish to track:

=> Metrics:
====> valid-views / views
====> actioned-views / views

=> Dimensions:
====> content dimensions
====> user demographics

** Valid views - views with minimal view time of 3 seconds
* Actioned views - views with reactions/likes/comments/shares

Question 1:
What would be the key entities in this model?

- User
- Content
- Device
- Location
- Event

Question 2:
What would be the key attributes for each entity? (provide up to 5 key attributes)

User - type - language - age - gender - signup_date - origin_country	Device - make - model - version
Content - type - duration - creation_date - language - origin_country	Location - country - region - city
	Event - event_type - event_value - event_start_ts - event_end_ts

Question 3:

How would you design the data model to support the entities and attributes?
What are the key tables you would build and what type of relationship exist between them (i.e 1:1, M:M)?.

```
dim_user
- user_id (pk)
- type
- language
- location (fk)
- origin_country
- age
- gender
- signup_date
- date (snapshot
partition)

dim_content
- content_id (pk)
- creator_id (fk)
- original_creator_id
- type
- duration
- creation_date
- language
- origin_country
- date (snapshot
partition)

dim_device
- device_id (pk)
- make
- model
- version
- date (snapshot
partition)

dim_location
- location_id (pk)
- country
- region
- city
```

```
fact_event (grain = 1 event per user per
content)
- event_id (pk) surrogate key
- user_id (fk)
- content_id (fk)
- device_id (fk)
- session_id
- event_type ( view, like, reply, repost )
- event_value
- event_start_ts
- event_end_ts
- date (incremental partition)
```

```
dim -> fact: 1:M
fact -> dim: 1:1
```

Question 4 (If ~3 mins remaining):

Some of the content presented in Newsfeed is shared content (i.e., user 1 shared a post created by user 2, and it's presented on user 3's Newsfeed), and it can be re-shared multiple times. How would you adjust your data model to allow for answering questions related to shared content? For example, deriving what the total number of reactions across all shared content for a given day is.

fact_event (grain = 1 event per user per content)

- event_id (pk) surrogate key
- user_id (fk)
- content_id (fk)
- device_id (fk)
- session_id
- event_type
- event_value - 1, 1, text, user_id
- event_start_ts
- event_end_ts
- is_shared - T/F
- date (incremental partition)

SQL

fct_story_interaction_di

- Date (partition)
- User_id
- Session_id
- Story_id
- Action_id
- Action_type

(view / reply / react / screenshot / report / share)

- View_start (DATETIME)
- View_end (DATETIME)

dim_story

- Date (partition)
- Story_id
- Story_type (photo / video / text)
- Creator_id
- Creation_date
- Story_length_seconds

(length of the story in seconds, NULL for photos)

- Last_update_date

Question 1 (Base):

Limited to a single date (today); Considering video media specifically, what percentage of video views reach the full duration. A full duration view is defined as from video start to video end. For the sake of convenience, we may ignore intermittent activity between video start and video end. In addition to the above percentage, calculate the total time spent watching videos, calculate the average length of videos views (average amount of time videos are watched).

1. percentage of video views reach the full duration
2. calculate the total time spent watching videos
3. average amount of time videos are watched

num: video views that reached the full duration today
den: video views today

A left join would include all content generated today even the ones with 0 views. NULL for duration sec

```
WITH content_today AS (
    SELECT
        content_id,
        video_length_seconds
    FROM dim_content
    WHERE date = current_date AND content_type='video'
)
fct_views AS (
    SELECT
        content_id,
        EXTRACT(EPOCH FROM view_end-view_start) AS duration_seconds
    FROM fct_newsfeed_action
    WHERE date=current_date AND action_type='view'
)
SELECT
    100.0*SUM(CASE WHEN fct.duration_seconds=ct.video_length_seconds
    THEN 1 ELSE 0 END)/COUNT(ct.content_id) AS pct_full_duration,
    SUM(fct.duration_seconds) AS total_watch_time,
    AVG(fct.duration_seconds) AS avg_watch_time
FROM content_today ct
JOIN fct_views fct ON ct.content_id=fct.content_id
```

Question 2: Story Engagement Analysis

What percentage of stories generated today had:

- At least one reaction today
- Zero replies today

Definitions:

- Numerator:

Number of stories generated today that had a reaction today but no replies today

- Denominator:

Total number of stories generated today

```
WITH content_today AS (  
    SELECT  
        content_id  
    FROM dim_content  
    WHERE date=current_date AND creation_date=current_date  
) ,  
fct_reactions_comments AS (  
    SELECT  
        content_id,  
        SUM(CASE WHEN action_type='react' THEN 1 ELSE 0 END) AS total_reactions,  
        SUM(CASE WHEN action_type='comment' THEN 1 ELSE 0 END) AS total_comments  
    FROM fct_newsfeed_action  
    WHERE date=current_date AND action_type in ('react', 'comment')  
    GROUP BY 1  
)  
SELECT  
    ROUND(100.0*SUM(CASE WHEN fct.total_reactions > 0 AND fct.total_comments=0  
THEN 1 ELSE 0 END)/COUNT(ct.content_id), 2) AS pct_reaction_no_comment  
FROM content_today ct  
LEFT JOIN fct_reactions_comments fct ON ct.content_id=fct.content_id
```

here the goal is to calculate for all content generated today even with 0 reactions and 0 comments. Doing an inner join would eliminate those from result set

Part 1: Data Definitions & Input

There is a real-time stream of events containing data about content viewed by a single user. Each event contains the following mandatory fields:

- `post_id`: Id of post viewed by user
- `viewed_time_ms`: Time spent on viewing given post (in milliseconds)
- `engaged_with_post`: Number of engagements (reactions, comments, etc.) with given post

Some events may come from content we run internal tests on. Those events will have one additional field:

- `is_test_content`: True if there is an internal test being run on given post

```
stream = [  
  {  
    'post_id': 101,  
    'viewed_time_ms': 6500,  
    'engaged_with_post': 1  
  },  
  {  
    'post_id': 104,  
    'viewed_time_ms': 200,  
    'engaged_with_post': 1  
  },  
  {  
    'post_id': 105,  
    'viewed_time_ms': 4200,  
    'engaged_with_post': 0,  
    'is_test_content': True,  
  },  
  {  
    'post_id': 108,  
    'viewed_time_ms': 4499,  
    'engaged_with_post': 1  
  },  
  {  
    'post_id': 105,  
    'viewed_time_ms': 500,  
    'engaged_with_post': 1  
  },  
]
```

Part 2: Problem Description & Requirements

A "queue" is a buffer/cache that has a fixed size where new items must wait for the oldest items to be removed before being added in. Assuming a buffer can hold only 3 events at a given time, write a function that can calculate and output the number of engagements, total time spent (in seconds), and the post ids in the buffer every time the buffer is full.

Formatting Requirements:

Each time metrics are computed, print the following formatted message:

Notes:

- Metrics should be computed only when the buffer is full.
- The function should assume that the buffer size of 3 and list of events can change.
- While computing metrics, you should ignore posts that have internal tests associated with them.

Expected output (for a buffer size of 3):

- You've got 2 engagements and spent 6.7s viewing content. Post ids: 101, 104
- You've got 2 engagements and spent 4.699s viewing content. Post ids: 104, 108
- You've got 2 engagements and spent 4.999s viewing content. Post ids: 108, 105

```
def process_stream(stream, buffer):  
    events = []  
    for event in stream:  
        events.append(event)  
        if len(events) == buffer:  
            engagements = 0  
            time_spent = 0  
            post_ids = []  
            for event in events:  
                engagements += event['engaged_with_post']  
                time_spent += event['viewed_time_ms']  
                post_ids.append(event['post_id'])  
            time_spent = time_spent / 1000  
            post_ids = ' '.join(post_ids)  
            print(f'You've got {engagements} and spent  
                {time_spent} s viewing content. Post ids: {post_ids}')  
            event.pop()
```


Set - 3

Feb 12th 25

Uber carpool

Product Sense

A rideshare service allows customers to use an app to request a driver to pick the customer up and drop them off at their destination. To reduce costs and the frequency of required trips, customers can select a 'carpool' option, which will overlap their route with other customers, resulting in an overall longer trip with multiple pick-ups and drop-offs

Common considerations when a carpooling algorithm is deployed:

Type of vehicle and maximum occupancy

Historical demand of trips along original route

Customer ratings

Value Proposition to Customer (how much is carpooling incentivised)

Let's say we recently launched this carpool option in the last couple of months. The executives want to know if the option is increasing app usage, and if the value proposition passed along to the customer is high-enough to encourage usage, without eating into the company's profits

Clarifying questions -

1) What business questions would help us understand the value proposition offered by the carpooling option? (10-12)

Rider

- Is the avg fare per trip decreased ?
- Are users saving money with the new feature ?
- Is the avg wait time similar to the regular trips ?
- Are the users satisfied with the new feature ?

Driver

.

- Is the idle time reduced ?
- Is drivers hourly earnings increase ?
- Are drivers completing more trips ?
- Is the driver satisfaction higher or lower ?

Company

- Is the company generating more revenue with this new feature ?
- Is the profit per trip increased ?
- Are we gaining or losing riders/drivers to the service ?
- How is the app engagement increase with this new feature ?

.

- 2) list the metrics that would answer the BQ's above ? (10-12)
- 10 or 12 metrics

Rider

- Avg Fare price Per trip -- regular or carpool rides?
- Avg Cost Savings per Carpool Trip
- NPS score
- Rider Retention /churn rate

Driver

- Avg earnings per hr
- Avg Idle Time Between Rides
- Driver Retention rate
- Avg weekly earnings per driver

Company

- revenue per carpool trip
- Profit Margin
- New User sign-ups per week (driver/rider)
- Churn Rate drivers/riders
- Avg carpool trips per ride
- pct of cancellation per total carpool bookings
- DAU/WAU/MAU
- ARPU - Avg Rev per users
- Total amount of promotions provided in past 7 days

3) What are the most important dimensions / cuts that would be relevant to metrics ?

Driver/Rider

- Age
- Gender
- sign-up date
- Location <-- city, country

vehicle

- Type
- Model
- Make
- Year
- Registration Expiry Date

Ride dimensions

- Trip timeframe (short, medium)
- Number of passengers
- Ride type (regular, carpool)
- No of stops

User Behaviour

- New vs Returning
- Ride frequency

Time Periods

- Time of Day
- Day of the week
- Seasonality/ Holiday

Data Modeling :

We would like to design a data model to be able to answer our business questions (How often is carpooling selected compared to regular rides, what is the average amount saved by the company through carpooling, what is the average cost passed on to the customer)

With this data model, we wish to track:

- => Metrics:
====> trips with carpool vs all rides
====> costs saved by carpooling vs price incentives to customer

Key Entities

Rider
Driver
vehicle
Location
Trip

Attributes

- Rider
- Age
 - gender
 - signup date
 - city
 - rating score
- Driver
- Age
 - gender
 - signup date
 - city
 - rating score
- vehicle
- type
 - Make
 - model
 - year
 - capacity
- Location
- region
 - city
 - country
 - state
 - zip code
- Trip
- pickup/dropoff time
 - prickup/dropoff location
 - fare
 - status

dimensions / Facts

dim_rider

- rider_id (PK)
- age
- gender
- signup_date
- city
- avg_rating_score
- date

dim_driver

- driver_id (PK)
- age
- gender
- signup_date
- city
- avg_rating_score
- dl_number
- date

dim_vehicle

- vehicle_id (PK)
- type
- make
- model
- year
- capacity
- registration_expiry_date
- date

dim_location

- location_id (PK)
- region
- city
- country
- state
- zip_code
- date

fct_carpool_trips

- trip_id (PK)
- driver_id
- rider_id
- location_id
- vehicle_id
- trip_placed_ts
- driver_acceptance_ts
- pickup_ts
- pickup_loc
- dropoff_ts
- dropoff_loc
- cancellation_reason
- ride_type (regular/ carpool)
- date

Relationships

Dims -> Fact (1:M)

Fact -> Dim (1:1)

- Learn to explain relationships

- Learn snapshot & incremental partition

Follow-up questions -

1. Why Driver and Rider cannot be in a single table?

2, What if you wanted to find out all the drivers associated with a vehicle?

VH01 -

ABC1

ABC2

dim_driver

- driver_id (PK)
- age
- gender
- signup_date
- city
- avg_rating_score
- dl_number

dim_vehicle

- vehicle_id (PK)
- drivers (list of drivers)
- type
- make
- model
- year
- capacity
- registration_expiry_date

Also talk about bridge table dim_vehicles_drivers

3. average number of riders per carpool rides?

fct_carpool_trips

- trip_id (PK)
- carpool_trip_id (parent)
- driver_id
- rider_id
- location_id
- vehicle_id
- trip_placed_ts
- driver_acceptance_ts
- pickup_ts
- pickup_loc
- dropoff_ts
- dropoff_loc
- cancellation_reason
- ride_type (regular/ carpool).

4. If we wanted to track the performance and not be impacted by the huge table size of FCT table? How do we do that

```
agg_rider_stats
agg_rider_stats
- region
- region
- vehicle_type
- vehicle_type
- date
- date
- age_group
- age_group
- average number of riders per hour
- average number of riders per hour
```

5. Avg savings per carpool ride

```
fct_carpool_trips
- trip_id (PK)
- carpool_trip_id (parent) --
- driver_id
- rider_id
- location_id
- vehicle_id
- trip_placed_ts
- driver_acceptance_ts
- pickup_ts
- pickup_loc
- dropoff_ts
- dropoff_loc
- regular_fare <-- 100
- carpool_fare <---- 60
- cancellation_reason
- cancelled_by
- ride_type ( regular/ carpool)
```

SQL

Table 1: fct_rides

column_name	data_type
ride_id	varchar
user_id	bigint
driver_id	bigint
vehicle_id	bigint
ride_type	varchar (values: 'carpool'/'regular')
region	varchar
start_time	timestamp
end_time	timestamp
date	varchar

Sample Data

ride_id	user_id	driver_id	vehicle_id	ride_type	region	start_time	end_time	date
abc123	1000	2001	1	carpool	NYC	2022-01-01 09:08:00	2022-01-01 09:08:00	2022-01-01
def456	1001	2002	2	regular	LA	2022-01-01 10:14:00	2022-01-01 10:14:00	2022-01-01
ghi789	1000	2003	1	carpool	NYC	2022-01-01 10:17:00	2022-01-01 10:17:00	2022-01-01

Table 2: dim_vehicles

column_name	data_type
vehicle_id	bigint
type	varchar
capacity	bigint

Sample Data

vehicle_id	type	capacity
1	sedan	3
2	SUV	6

1) Ratio of carpool:all trips Iin last 30 days

```
SELECT COUNT(CASE WHEN ride_type = 'carpool' THEN ride_id ELSE
NULL END) * 1.0 / COUNT(*) AS ratio_carpool_trips
FROM fct_rides
WHERE date > (current_date - 30)
```

2) Rank the Type of Vehicles by Most Used for pool, ranking should be done based on customer timespent in the vehicle ?

```
SELECT
dim.type,
COALESCE(SUM(EXTRACT(EPOCH from end_time - start_time)),0) as time_spent_on_pool
FROM
    (SELECT vehicle_id,type FROM dim_vehicles) dim
LEFT JOIN
    (SELECT vehicle_id, start_time, end_time FROM fct_rides where ride_type =
'carpool') fct
    ON dim.vehicle_id = fct.vehicle_id
GROUP BY dim.type
ORDER BY COALESCE(SUM(EXTRACT(EPOCH from end_time - start_time)),0) DESC
```

3) Number of drivers who made more Pool Rides than Regular Rides

```
WITH driver_stats AS (
SELECT
    driver_id,
    SUM(CASE WHEN ride_type = 'carpool' THEN 1 ELSE 0 END) as num_pool_rides,
    SUM(CASE WHEN ride_type = 'regular' THEN 1 ELSE 0 END) as num_regular_rides
FROM fct_rides
GROUP BY driver_id
)

SELECT count(distinct driver_id) AS total_drivers
FROM driver_stats
where num_pool_rides > num_regular_rides
```

4) Create a query to generate an aggregate table that will power a dashboard. On this dashboard, we will need to show the ratio of carpool trips by region,vehicle type, and date.

```
SELECT
    region,
    vehicle_type,
    date,
    SUM(CASE WHEN ride_type = 'carpool' THEN 1.0 ELSE 0 END) * 1.0 / COUNT(*)
AS ratio_carpool_trips
FROM fct_rides AS fct
JOIN dim_vehicles AS dim ON fct.vehicle_id = dim.vehicle_id
GROUP by Grouping sets (
    (1,2,3), (1,2), (2,3), (1,3), (2), (3), (1), ()
)
```

Python

Python :

You will be given total passenger capacity in a vehicle, list of trips - write a python function - , return T/F whether that booking_requests is possible by driver or not

Input Example:

booking_requests - [{request_id, pickup_time, drop_off_time, total_passenger},]

Example vehicle passenger capacity = 6

```
booking_requests = [
    {"request_id": 1, "pickup_time": 10, "drop_off_time": 20, "total_passengers": 3},
    {"request_id": 2, "pickup_time": 15, "drop_off_time": 25, "total_passengers": 2},
    {"request_id": 3, "pickup_time": 30, "drop_off_time": 40, "total_passengers": 2},
    {"request_id": 4, "pickup_time": 35, "drop_off_time": 45, "total_passengers": 3},
    {"request_id": 5, "pickup_time": 50, "drop_off_time": 60, "total_passengers": 4},
    {"request_id": 6, "pickup_time": 55, "drop_off_time": 65, "total_passengers": 4},
    {"request_id": 7, "pickup_time": 70, "drop_off_time": 80, "total_passengers": 5},
    {"request_id": 8, "pickup_time": 75, "drop_off_time": 85, "total_passengers": 3},
    {"request_id": 9, "pickup_time": 90, "drop_off_time": 100, "total_passengers": 1}
]
```

```
def booking_process(booking_requests, capacity):
    current_list = []
    for request in booking_requests:
        pickup = request['pickup_time']
        dropoff = request['drop_off_time']
        passengers = request['total_passengers']
        current_list.append((pickup, passengers))
        current_list.append((dropoff, -passengers))
    current_list.sort(key: lambda x: x[0], x[1])

    current_passengers = 0
    for time, count in current_list:
        current_passengers += count
        if current_passengers > capacity:
            return False
    return True
```

Product Sense

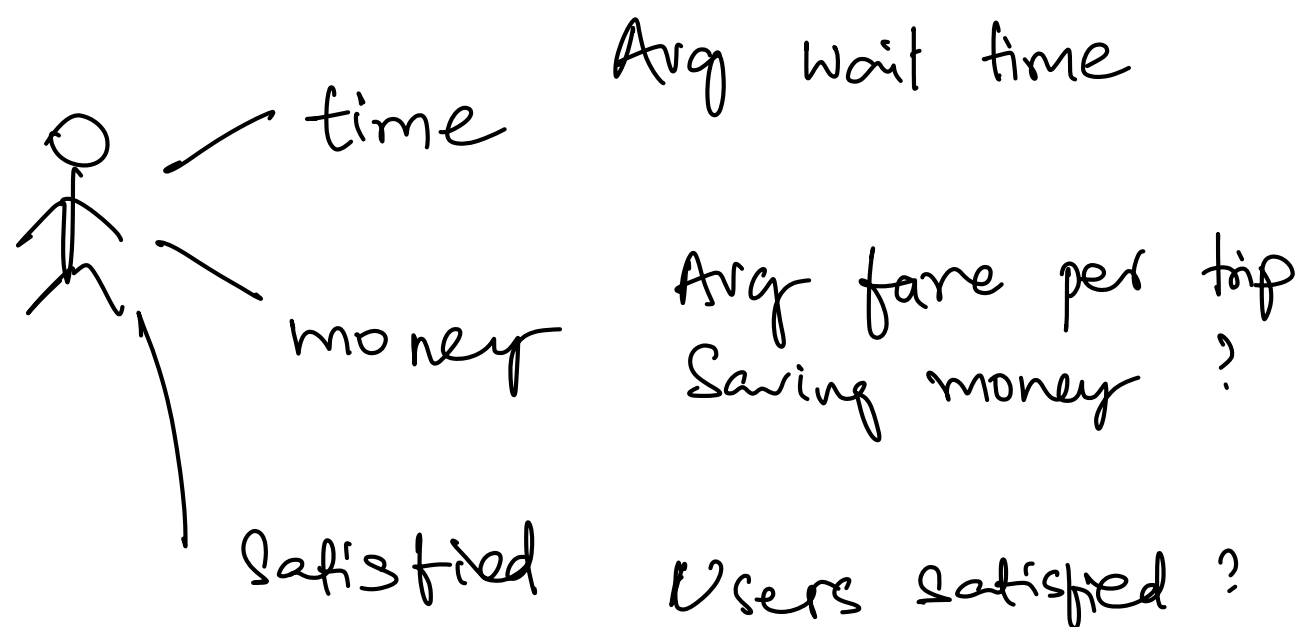
① Value Proposition to the business with the new car pool feature in Uber

Clarifying Questions -

1. Which markets has this launched in?
2. Is the goal to understand impact on the drivers, riders and Company?

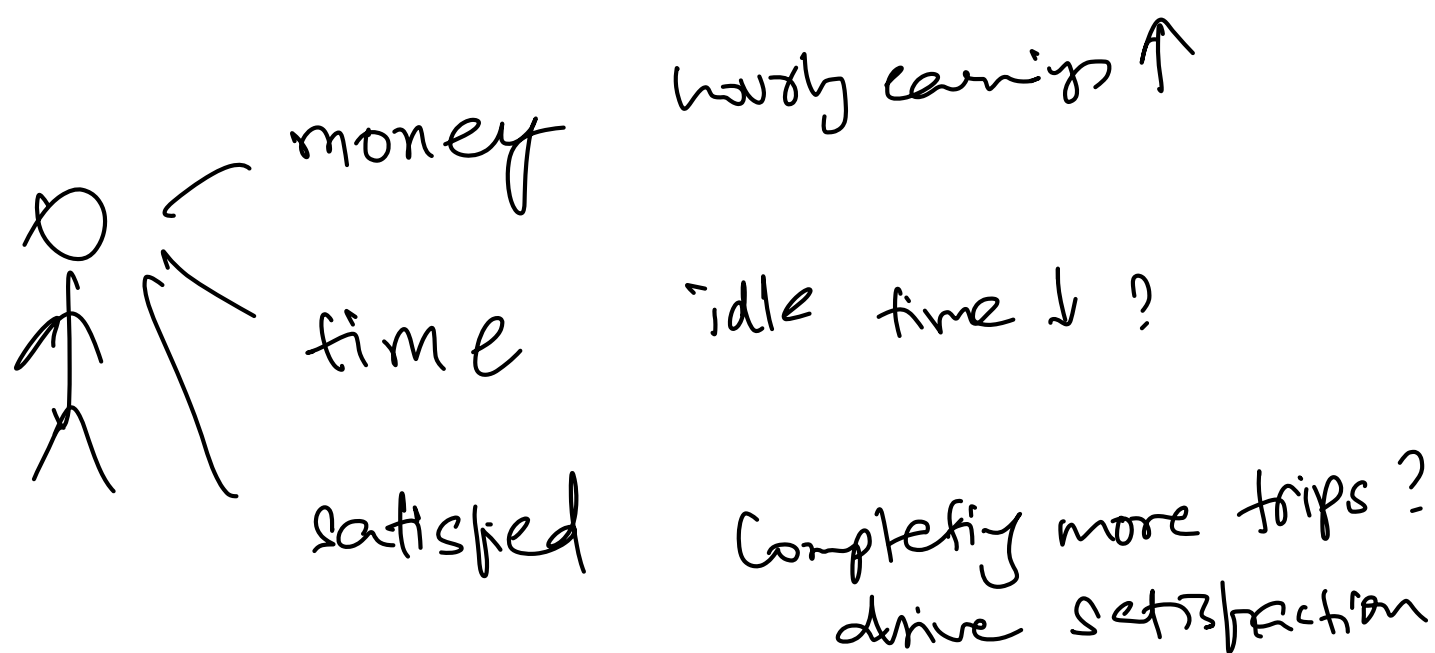
Value Proposition Question

Rider



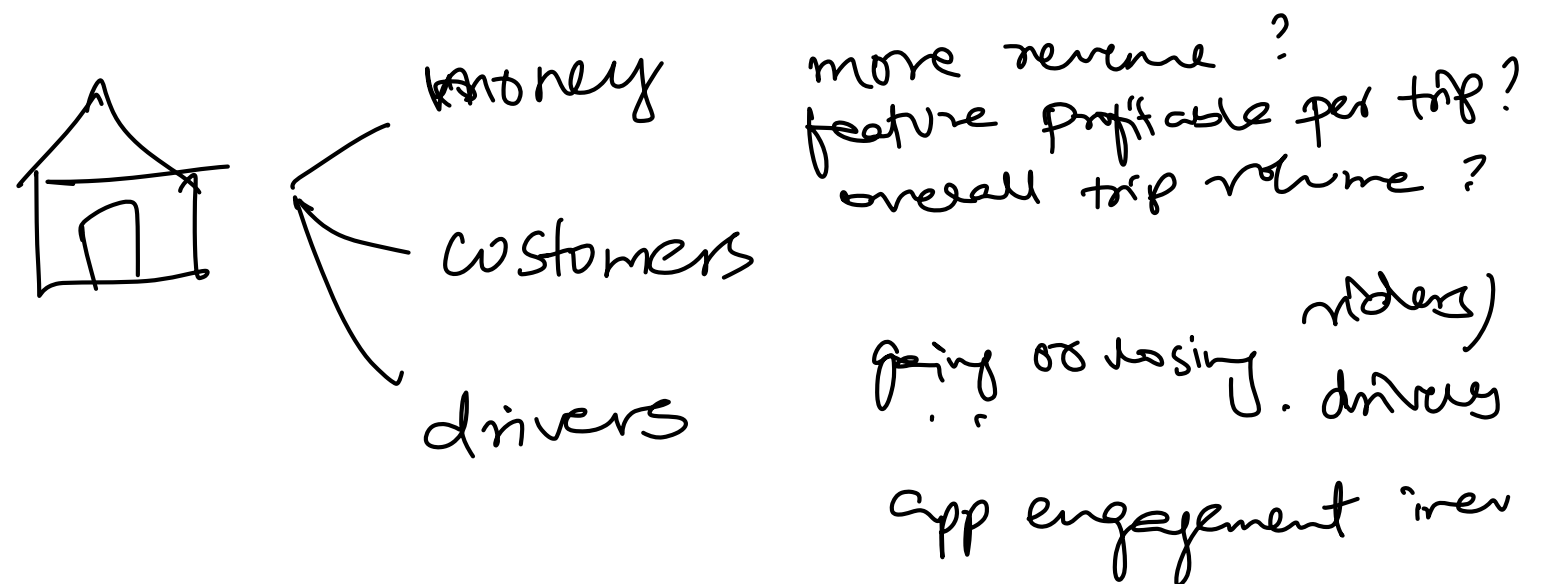
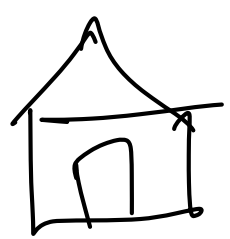
1. Is the avg fare per trip decreased comparatively?
2. Are users saving money with this feature?
3. Is the avg wait time similar?
4. Are users satisfied higher or lower?

Driver



1. Is the idle time reduced?
2. Is drivers hourly earnings increased?
3. Are drivers completing more trips?
4. Is driver satisfaction higher or lower?

Company -



1. Is uber generating more revenue with new feature?
2. Is the profit per trip increased?
3. Is the overall trip volume increase?
4. Did we gain or lose riders/drivers to the app?
5. Is the app engagement increase with new feature?

Metrics

Driver

1. Avg fare per trip
2. Avg earning per hour
3. Avg idle time per hour
4. Avg detour time per trip
5. Carpool acceptance rate
6. Driver retention rate
7. Avg no. of passengers per trip
8. Avg distance driven per trip

Rider

1. Avg wait time
2. Avg trip time
3. Avg fare price per trip vs solo trips
4. Cost savings % per trip
5. Wait time for carpool vs solo
6. Rider satisfaction (ratings for carpool trips)
7. Carpool feature adoption rate
8. Frequency of carpool feature per rider
9. Net promoter score for app
10. Cancellation rate for carpool

Drivers :-

Avg earnings per hr
trips completed per shift
Idle or wait time b/w trips
driver satisfaction ratings
Driver retention rate
Avg occupancy rate per trip
Acceptance rate
Net earnings

Company

Total carpool trips completed
Revenue per carpool trip
Profit margin
Overall platform growth
Driver / Rider Acquisition
Churn rate driver / rider
Session engagement app opens, feature usage

User growth rate month over month

Dimensions / Cuts

Ride Dimensions

Trip timeframe (>30, >40 min)

Number of People (1, 2, 3, ...)

Ride type (regular, Pool)

Driver/Customer Demographics

Age

Gender

Location

User Behavior :

New vs Returning

Ride frequency

Vehicle Dimensions :

Type

Model

Make

year

Category

Platform :

Device type

OS

Time Periods

Time of Day

Day of the week

Seasonality

Data Modeling

Q:- How often are delivery pool orders selected compared to regular orders?

Avg amount saved by the company through pool deliveries?

Avg cost passed on to the customer?

Delivery Pool orders vs Regular orders

Costs saved " vs Price incentives

Q1 :: Key Entities & Attributes

Business Process

1. Customer (Rider) requests for a trip. There are 2 options pool / regular
2. Driver comes to the location of Driver, picks them up in his vehicle and drops at desired location
3. Driver also has option to pick regular/pool trip
4. trips can be cancelled by driver or rider at any stage for any reason.

Key Entities

Rider

Driver

Vehicle

Location

Device

Rider → age, gender, signup date, rating score, location

Driver → age, gender, signup date, rating score, location

Vehicle → type, make, model, year, capacity

Location → city, county, state, country, zip

Device → OS, device type, version, browser type

Trip → pickup time, wait time, status, fare

fact-carpool-trips

trip_id (PK)

driver_id

rider_id

vehicle_id

trip_placed_ts

driver_acceptance_ts

driver_arrived_loc_ts

rider_pickup_ts

pick up_location

trip_completed_ts

drop-off_location

status

cancellation_reason

ride type (regular/pool)

dim-driver

driver_id

age

gender

Country

Signup date

Avg-rating-score

dim-rider

customer_id

age

gender

Signup date

Avg rating score

dim-vehicle

vehicle_id

" -type

" -capacity

vehicle plate-expiry-date

make

model year

dim-location

location_id

-type

address

city

state

Country

dims $\rightarrow$ Fact

Fact $\rightarrow$ dim

1:M

1:1

bridge tables

drivers $\rightarrow$ vehicles M:M

SQL

fact-deliveries

delivery-id
driver-id
vehicle-id
customer-id
delivery-type
region
pickup-time
drop-off-time
date

dim-vehicles

vehicle-id
type
order_capacity

Ratio of pooled deliveries to all deliveries in last 30 days

num $\rightarrow$ #Pooled deliveries

All deliveries

Sum(
Select case when delivery_type = 'pool' then 1 else 0
From fact-deliveries end) / count(*)
Where date $\geq$ (current-date - 30)

2. No. of drivers who made more pooled deliveries than

driver stats ~~regular~~ drivers

select

driver_id

Sum(case when delivery_type = 'pool' then 1 else 0 end)

From fact-deliveries

Group by driver_id

= 'regular'

as pool-deliveries

select count(distinct driver_id)

from driver_stats

where pool-deliveries > regular-deliveries

3. Rank the type of vehicles most used for pooled deliveries?

delivery-type = 'pool'
of deliveries
group by vehicle type

Total time
spent
on
deliveries

select type,
count (delivery-id) as # deliveries

from fct-deliveries
left join dim-vehicles on vehicle-id
where delivery-type = 'pool'
group by type
order by #.deliveries desc
limit 1

select dim.type
from (select vehicle, type from dim-veh) as dim
left join (select vehicle, pickup, time, drop off
time where de = 'pool') as fct
on dim-veh = fct-
group by type
order by coalesce (extract (epoch from
drop off time - pickup time), 0) desc

4 Write a query to generate agg table for dashboard.

region
vehicle-type
delivery-date
ratio of pooled deliver to all

select

region

type as vehicle-type

date as delivery-date

sum(as ratio)

from fact
join dim

on vehicle-id

group by

grouping sets (

(1,2,3), (1), (2), (3),

(1,2), (2,3), (3,1), (1))

rate?

manuliv?

do we need

multiple groups

score?

Python

① booking_requests = [

{ 'request-id'
pickup time
drop off time
2] total passengers

capacity = 6

Total passenger capacity in vehicle

list of trips

return T/F — booking_requests possible or no

	3	2	2	3	4	4	5	3	1					
Time	10	15	20	25	30	35	40	45	50	55	60	65	70	
Passengers	3	5	2	0	2	5	3	0	4	8	4	0	5	
result	T	T	T	T	T					X				

So False

min pickup time

max drop off time

def booking_cab (booking_request, capacity):

current_list = []

for request in booking_requests:

pickup = request ['pickup-time']

passengers = request ['curr-q-pass']

dropoff = request ['dropoff-time']

```
current_list.append((pickup, passenger))  
current_list.append((dropoff, passenger))  
current_list.sort(key = lambda p: p[0], p[1])
```

```
current_passengers = 0
```

```
for time, change in current_list:  
    current_passengers += change  
    if cur_ " > capacity:  
        return False  
return True.
```

Set-4

Door Dash

We recently launched a grocery delivery app similar to Instacart. this app allows customers to order grocery items for pick up or delivery within within the app. There is an interface for customers to browse the product catalogue. Select their store place orders, and choose the delivery method pick up or delivery.

Clarifying questions -

1. did the app launch in a specific region or is it a country wide launch
2. do we have any premium plans or is it free plan for everyone?

As part of the analytics team, we need to evaluate how different stores are performing on the app what metrics and dimensions should we consider?

Revenue metrics

- Gross merchandise value per store - total dollar value of all orders before any deductions
- Average order value - $\text{total revenue} / \text{total orders}$
- Revenue per store - actual revenue store receives after discounts from emotional codes or fees
- Revenue growth rate - $\text{percentage change in revenue over a time. Period. } (\text{Current period revenue} - \text{previous period revenue}) * 100 / \text{previous period revenue}$

Order metrics

- Total number of orders
- Order completion rate - $(\text{completed orders} / \text{total orders placed}) * 100$
- Cancelled order rate - $(\text{cancelled orders} / \text{total orders}) * 100$
- Order frequency - orders per day/ orders per hr . Higher velocity means more consistent demand.
- Refund rate - percentage of orders refunded

Customer metrics

- Number of unique customers per store
- New customer acquisition rate
- Repeat Customer rate - $(\text{customers with 2+ orders} / \text{total unique customers}) * 100$
- Customer retention rate - proportion of returning customers among existing ones
- Customer churn rate - rate at which customers stop ordering from platform

Quality metrics

- CSAT - average score on overall satisfaction level scale 1-5
- NPS - measures loyalty of Customer recommending to a friend or the store. Promoters - detractors.
- App ratings specific stores

User

- Age
- Gender
- Location
- Signup date
- Email
- New vs Returning customers
- Free vs Premium subscribers

Location

- Region
- State
- Urban vs Suburban vs rural
- Population density zone

Order

- Type (pickup/delivery)
- Size (small,medium,large)
- Value
- First time vs repeat orders

Store

- Type (grocery/pharmacy)
- Size (small/M/L)
- Store chain/brand
- Premium vs Standard stores

Time

- Time of Day
- Day of week
- Seasonality / holidays

The app has been launched for a few months of a metrics are showing that average revenue per store is trending down week over week. How would you triage the root cause of this decline?

Clarifying questions

- Is the decline universal or localized?
- When did the decline start?

Confirm on the metric -

Average Revenue per store = Total Revenue / Total Stores

- Decreasing the number of orders or order value
- Increase in the number of stores
- Both revenue has decreased and number of stores might have been increased at the same time

Debugging

Analyze pipelines and upstream sources

- DQ issues like Null, missing data
- Logic issues - incorrect joins, filters

Decompose revenue into components

- Revenue = number of orders * average order value
- Is the order volume decreasing ?
- Is average order value decreasing ?

Check if new restaurants have been increased with the locality

- Total new restaurants over time

Breakdown revenue by different cuts on the region, restaurant type, order type to see if there's any correlation with the drop

- Average Revenue per each breakdown category

Internal factors

- New feature launch - check metrics before and after launch
- New version launch - breakdown by version type
- Bugs - From code releases - crash rate
- UI changes

External factors

- Competition - discounts by competitors
- Global issues
- Seasonality or holidays

Check whether users are entering the app

- Metrics to check - DAU

To encourage customer loyalty and drive more uses to the app? What product features would you suggest the team to implement?

Retain existing customers

- loyalty & reward programs like cash backs
- Improve experience with quality, assurance & responsive support
- improve UI
- easier order
- improve tracking order
- Referral discount
- faster deliver

Attracting new customers

- referral programs
- first order promotions
- paid advertising
- reward points
- coupons
- Offers

Data Modeling

Design a design, a data model for Instacart, focusing specially, on the stores below the questions that needs to be answered using the Data model

- 1.How many orders are placed and from which stores?
- 2.What's the average order size and value?

Clarifying Process

Business Process -

dim_store	Dim_customer	Fct_orders
Store ID	Customer_ID	Order id
Name	Name	Store id
Category	Email	Product id
Location	Phone number	Customer id
Delivery available	Signup date	Order type (pickup, delivery)
Phone number	Age	Order accepted TS
Owner name	Gender	Order ready TS
Date	date	Drive id
		Driver assigned ts
		Driver reached ts
		Driver delivered ts
		Order status
		Cancellation ts
		Cancellation reason
		Total items
		Total order cost
		App fee
		Driver fee
		Store net amount
		date

Avg order size = total no of orders / total items ordered

Avg order value = total cost of all orders/ total no of orders

Single store can have multiple product categories example of different sections in grocery store with weighing items. How do you incorporate this in your data modeling?

To address this we modify the product dimension to handle different categories of products example fruits, vegetables, beverages, and to reflect this same category dimension, but there can be multiple categories for a single product same bread as (bakery, snack) yogurt has (diary or organic produce) chicken as (meat, frozen foods)

bridge table

Dim product category

Category product id

Product ID

Category id

Category type

Store ID

Name

Size

Price

Ingredients

Date

1 product → multiple category

1 category → multiple products

(M:M)

SQL

```
dim_users
-----
user_id
name
phone
email
registration_date
city
account_plan      (e.g. 'free', 'premium')

dim_stores
-----
store_id
name
address
city
state
zip_code
category          (e.g. 'produce', 'dairy')

-- Fact table for orders
fct_order
-----
order_id          PK
store_id          FK
driver_id         FK
user_id           FK
order_type        (e.g. 'pickup', 'delivery')
net_order_amount
date              (PARTITION)
```

Question 1: What is the average order value (\$) per user's city today?

```
.SELECT
    u.city,
    SUM(o.net_order_amount) / COUNT(o.order_id) AS avg_order_value
FROM fct_order o
JOIN dim_users u ON o.user_id = u.user_id
WHERE o.date = CURRENT_DATE
GROUP BY u.city
ORDER BY avg_order_value DESC;
```

Question 2: How many stores do more pickups than deliveries today?

```
WITH order_counts AS (
    SELECT
        store_id,
        SUM(CASE WHEN order_type = 'pickup' THEN 1 ELSE 0 END) AS
pickup_count,
        SUM(CASE WHEN order_type = 'delivery' THEN 1 ELSE 0 END) AS
delivery_count
    FROM fct_order
    WHERE date = CURRENT_DATE
    GROUP BY store_id
)
SELECT
    COUNT(*) AS stores_with_more_pickups
FROM order_counts
WHERE pickup_count > delivery_count;
```

Question 3: What are the three most popular categories in the last month?

```
SELECT
    s.category,
    COUNT(DISTINCT o.user_id) AS unique_customers
FROM fct_order o
JOIN dim_stores s ON o.store_id = s.store_id
WHERE o.date >= CURRENT_DATE - INTERVAL '1 month'
    AND o.date < CURRENT_DATE
GROUP BY s.category
ORDER BY unique_customers DESC
LIMIT 3;
```

Top 3 Store Categories with Revenue Share

```
WITH total_revenue AS (
    SELECT SUM(net_order_amount) AS total_revenue
    FROM fct_order
    WHERE cancelled_by IS NULL
        AND date >= CURRENT_DATE - INTERVAL '1 month'
        AND date < CURRENT_DATE
)
SELECT
    s.category AS cuisine,
    COUNT(DISTINCT o.user_id) AS unique_users,
    SUM(o.net_order_amount) AS total_revenue,
    ROUND(100.0 * SUM(o.net_order_amount) / c.total_revenue,
2) AS revenue_percent_share
FROM fct_order o
JOIN dim_stores s ON o.store_id = s.store_id
CROSS JOIN total_revenue c
WHERE o.cancelled_by IS NULL
    AND o.date >= CURRENT_DATE - INTERVAL '1 month'
    AND o.date < CURRENT_DATE
GROUP BY s.category, c.total_revenue
ORDER BY unique_users DESC
LIMIT 3;
```

Python

Problem: Delivery Time Estimation

We want to provide customers with an estimate of when their order will be delivered. We have driving time between locations and a delivery routing sequence. Use that data to create a map of order_id to an expected delivery time message in the format:

"Order <order_id> is expected to be delivered at <time>."

Given Data:

You will be given a list of routing_steps with one of three actions:

- TRAVEL → The driver travels from the current location to the next location.
- PICKUP → The driver picks up an order at the current location.
- DROPOFF → The driver drops off the order at the current location.

```
routing_steps = [
    {"driver_id": 1, "action": "TRAVEL", "location_id": 2},
    {"driver_id": 1, "action": "PICKUP", "order_id": 1},
    {"driver_id": 1, "action": "TRAVEL", "location_id": 3},
    {"driver_id": 1, "action": "DROPOFF", "order_id": 1}
]

travel_time = [
    [ 0, 10, 17, 23, 9, 13 ],
    [10, 0, 30, 25, 5, 25 ],
    [17, 30, 0, 10, 7, 33 ],
    [23, 25, 10, 0, 11, 35 ],
    [ 9, 5, 7, 11, 0, 27 ],
    [13, 25, 33, 35, 27, 0 ]
]

routing_steps = [
    {"driver_id": 1, "action": "TRAVEL", "location_id": 1},
    {"driver_id": 2, "action": "TRAVEL", "location_id": 3},
    {"driver_id": 1, "action": "PICKUP", "order_id": 1},
    {"driver_id": 1, "action": "TRAVEL", "location_id": 2},
    {"driver_id": 2, "action": "PICKUP", "order_id": 2},
    {"driver_id": 2, "action": "PICKUP", "order_id": 3},
    {"driver_id": 2, "action": "TRAVEL", "location_id": 4},
    {"driver_id": 2, "action": "PICKUP", "order_id": 4},
    {"driver_id": 1, "action": "DROPOFF", "order_id": 1},
    {"driver_id": 1, "action": "TRAVEL", "location_id": 5},
    {"driver_id": 2, "action": "DROPOFF", "order_id": 4},
    {"driver_id": 2, "action": "DROPOFF", "order_id": 2},
    {"driver_id": 2, "action": "TRAVEL", "location_id": 1},
    {"driver_id": 2, "action": "DROPOFF", "order_id": 3}
]
```

```
Order 1- expected delivery time is 40
Order 2 expected delivery time is 61
Order 3 expected delivery time is 86
Order 4 expected delivery time is 61
```

```

def calculate_delivery_times(routing_steps, travel_time):

    # Track current location and cumulative time for each driver
    driver_state = {}

    # Track delivery times for each order
    order_delivery_times = {}

    for step in routing_steps:
        driver_id = step["driver_id"]
        action = step["action"]

        # Initialize driver state if not exists (starts at location 0
        # with time 0)
        if driver_id not in driver_state:
            driver_state[driver_id] = {
                "current_location": 0,
                "current_time": 0
            }

        if action == "TRAVEL":
            location_id = step["location_id"]
            current_location = driver_state[driver_id]
            ["current_location"]

            # Add travel time from current location to new location
            travel_duration = travel_time[current_location][location_id]
            driver_state[driver_id]["current_time"] += travel_duration
            driver_state[driver_id]["current_location"] = location_id

        elif action == "DROPOFF":
            order_id = step["order_id"]
            # Record the delivery time
            order_delivery_times[order_id] = driver_state[driver_id]
            ["current_time"]

    return order_delivery_times

result = calculate_delivery_times(routing_steps_1, travel_time)
for order_id, delivery_time in sorted(result.items()):
    print(f"Order {order_id} expected delivery time is {delivery_time}")

```


Set -5

Twitter (Followers / Followees)

Product Sense

We recently introduced private accounts to popular social networking platform.
For public account: anyone can follow and view content
for private account: requires an approved follow request to view content.

Since the launch of private accounts with a noticed overall engagement on the platform has declined. what are your next steps? How would you approach this? 🧐

Clarifying questions -

- How long has it been since launch and is engagement still declining or stabilizing?
- Did many high influence accounts go private
- What specific metrics declined is it D daily active users monthly active users

What steps would you take and how would you approach this?

Data Quality Issues

- Validate if the engagement drop is real if it's caused by the data pipe, failures are inaccuracies if yea fixing them is the first step

Bugs in the logic

- Review Weather Dera box in the private account implementation, which unintentionally restrict the contents visibility
- If issues exist, we can deploy the quick fixes

Content discovery problem

- Private content likely removed from the from the feed
- This creates low impressions per post

Slower Follower Growth

- In private account accounts as a request or manually approved, there could be a friction in the follower growth
- As content can't be reached beyond approved followers. Reduced reach will have reduced engagement loop.

User Composition Shift

- If if high follower accounts when private, there could be an impact on the engagement

Behavioral changes

- Private users, my post less frequently
- Loss of public validation reduces posting motivation

What metrics explain the engagement decline 📉

- Number of engagements (overall count of likes, comments, shares or requests)
- Average number of engagements per post
- Engagement rate (engagements relative to reach or impressions)
- Retention rate
- Churn rate
- Exchange rate
- Average follower growth rate (how quickly users gain new followers)
- Follower acceptance rate (percentage of follow requests approved)
- Average time to accept follow request (delay between request and approval)
- Daily percentage of accounts, switching from public to private and from private to public
- Percentage of private accounts versus public accounts of a platform

What are the most important dimensions/cuts for your metrics?

User Demographics

- Age
- Gender
- Location
- Race
- sign up date
- Origin country

Time Periods

- Time of the day
- Day of the week
- Weekday vs weekend
- Holiday vs workday
- Seasons

Types of Engagements

- Likes
- Shares
- Comments
- Saves

Types of Posts

- Reels
- Stories
- Photos
- Videos

Account types

- Public users
- Private users

Content creator cohort

- Influencers
- Regular users

Data Modeling

Business process

- Users can sign up to the platform (public or private)
- Create and share content
- Public or private for post
- Follow our users
- Engage with content like comment share, save

Dim-user

- user id
- Username
- Age
- Gender
- Language
- Location_id(fk)
- Created-at
- Last-active
- Account-type
(public / private)
- date (partition)

dim-content

- content id
- Content type
- Category
- user-id
- created at
- duration
- date (partition)

dim-device

- device-id (pk)
- device type
- os
- app version
- date (partition)

fct_event_interactions

Event-id
User id
Session-id
Action_type
(view, like, comment, share)
Action -content
Content-id
start_ts
end_ts
device_id
location_id
date (partition)

1.How do you get followers count ?

Add follower_count in the dim_user Table

2.How do you track who is following whom ?

One user can have multiple followers (1 user → M followers)

One follower can follow multiple users (1 follower → M users)

M:N relationship

Create a bridge table dim_user_follow

- user-id
- follower-id
- timestamp
- date (partition)

3.How do you track follow requests and acceptances ?

fct_user_follow

- follow_event_id (PK)
- user_id (FK)
- follower_id (FK)
- event (follow_requesr, follow_accept, unfollow etc)
- event_details
- event_ts
- date (partition)

4.How to track all the engagements across users ?

agg_user_engagement_summary

- user_id (PK)
- total_likes
- total_comments
- total_shares
- total_engagements
- date (partition)

SQL

Dim_user:

- user_id
- privacy_setting (private/public)
- privacy_setting_last_update_ts
- Date (Partition)

Fct_follows:

- Target_user (request acceptor/rejector based on event)
- Source_user (request sender)
- event/action (
- follow_request,
- follow_accept, (private profile needs to accept followers)
- follow_success, (public profile has a default acceptance)
- reject,
- unfollow)
- action_ts

Question 1) For the users who went private in the last 7 days, find out the overall acceptance rate for those users for last 7 days?

Overall acceptance rate is defined as: % of follow requests that have been accepted / total num of requests

Den: number of follow requests sent in last 7 days by people who turned private in last 7 days

filter - $\text{privacy_setting} = \text{private}$
 $\text{privacy_setting_last_updated} - 7 \text{ days}$

join on user_id to target-user

$\text{SUM}(\text{CASE WHEN follow_accept} = \text{TRUE THEN 1 ELSE 0 END})$
 $* 100 / \text{SUM}(\text{follow_request})$
 $\text{action_ts} - 7$

```
WITH users_went_private AS (  
  SELECT user_id  
  FROM dim_user  
  WHERE privacy_setting = 'private'  
        AND privacy_setting_last_update_ts >= CURRENT_DATE - 6  
        AND privacy_setting_last_update_ts <= CURRENT_DATE  
        AND date = current_date  
) ,
```

```
requests_to_private_users AS (  
  SELECT  
    f.target_user,  
    f.event_action,  
    f.action_ts  
  FROM fct_follows f  
  INNER JOIN users_went_private u  
    ON f.target_user = u.user_id  
  WHERE f.action_ts >= CURRENT_DATE - 6  
        AND f.action_ts <= CURRENT_DATE  
) .
```

```
SELECT  
  SUM(CASE WHEN event_action IN ('follow_accept', 'follow_success') THEN 1  
ELSE 0 END) * 100.0  
  / NULLIF(SUM(CASE WHEN event_action = 'follow_request' THEN 1 ELSE 0 END),  
0)  
  AS overall_acceptance_rate  
FROM requests_to_private_users;
```

Fct_follows:

- Source_user (request sender)
- Target_user (request acceptor/rejector based on event)
- event/action (
- follow_request,
- follow_accept, (private profile needs to accept followers)
- follow_success, (public profile has a default acceptance)
- reject,
- unfollow)
- action_ts

Question 2) Create the table dim_follows with the following schema based on fct_follows table.

Dim_follows: (description: table of who is following who on a certain date and from when is it valid)

- date(p)
- source
- target
- valid_from

```
CREATE TABLE dim_follows
PARTITIONED BY (date)
AS
```

```
WITH last_follow AS (
    SELECT
        source_user,
        target_user,
        MAX(action_ts) AS max_action_ts
    FROM fct_follows
    WHERE event_action IN ('follow_accept', 'follow_success', 'unfollow')
    GROUP BY
        source_user,
        target_user
)
```

```
SELECT
    CURRENT_DATE AS date,
    f.source_user AS source,
    f.target_user AS target,
    f.action_ts AS valid_from
CASE
    WHEN r.source_user IS NOT NULL THEN TRUE
    ELSE FALSE
END AS is_reciprocal
FROM fct_follows f
JOIN last_follow l
    ON f.source_user = l.source_user
    AND f.target_user = l.target_user
    AND f.action_ts = l.max_action_ts
LEFT JOIN last_follow r
    ON f.source_user = r.target_user
    AND f.target_user = r.source_user
WHERE f.event_action IN ('follow_accept', 'follow_success');
```

Q2: SQL - Add is_reciprocal Column

We want to add a column to the dim_follows table called is_reciprocal, which tells us if the source and target user have a reciprocal relationship.

Definition: A reciprocal relationship exists if User A follows User B AND User B also follows User A.

Input Data: dim_follows

date	source_user_id	target_user_id	follow_time
2024-09-30	101	202	2024-09-30 11:30:00
2024-09-30	202	101	2024-09-30 13:00:00
2024-09-30	103	204	2024-09-30 14:00:00
2024-09-30	107	210	2024-09-30 14:45:00

Expected Output:

date	source_user_id	target_user_id	follow_time	is_reciprocal
2024-09-30	101	202	2024-09-30 11:30:00	true
2024-09-30	202	101	2024-09-30 13:00:00	true
2024-09-30	103	204	2024-09-30 14:00:00	false
2024-09-30	107	210	2024-09-30 14:45:00	false

SELECT

 f.date,

 f.source_user_id,

 f.target_user_id,

 f.follow_time,

 CASE

 WHEN r.source_user_id IS NOT NULL THEN TRUE

 ELSE FALSE

 END AS is_reciprocal

FROM dim_follows f

LEFT JOIN dim_follows r

 ON f.source_user_id = r.target_user_id

 AND f.target_user_id = r.source_user_id

 AND f.date = r.date;

Find active connections between users at the end of each day. A user is considered to be actively following another if the last event of that day is a valid follow (follow_accept or follow_success).

Example Data: fact_follow_actions

event_id	source_user_id	target_user_id	event_time	event_name	entrypoint
101	202	2024-09-30	10:00:00	follow_request	profile_page
102	202	2024-09-30	11:30:00	follow_accept	notification
103	204	2024-09-30	12:45:00	follow_reject	direct_link
105	206	2024-09-30	14:00:00	follow_success	search
107	208	2024-09-30	15:15:00	unfollow	profile_page

Expected Output:

date	source_user_id	target_user_id	follow_time
2024-09-30	101	202	2024-09-30 11:30:00
2024-09-30	105	206	2024-09-30 14:00:00

```
SELECT
    DATE(event_time)                AS date,
    source_user_id,
    target_user_id,
    MAX(event_time)                AS follow_time
FROM fact_follow_actions
WHERE event_name IN ('follow_accept', 'follow_success')
GROUP BY
    DATE(event_time),
    source_user_id,
    target_user_id;
```

Python

Python Q1: Reciprocal Follow Count

Question:

Given a dictionary friends where each key is a user and the value is a list of users they follow, write a function to return the number of reciprocal follows for each user. A reciprocal follow means user A follows B and B follows A.

Sample Input:

```
friends = {
    'a': ['b', 'c', 'd'],
    'b': ['a', 'd', 'e'],
    'c': ['a', 'd', 'e'],
    'd': ['a', 'e', 'f'],
    'r': ['d']
}
```

1. Loop through each user
2. For each person they follow, check:
→ Does that person also follow them back?
3. Count how many do → that's the reciprocal count

Expected output:

```
{'a': 3, 'b': 3, 'c': 2, 'd': 4, 'e': 2, 'f': 2}
```

```
def reciprocal_follow_count(friends):
    result = {}

    for user, following_list in friends.items():
        count = 0

        for person in following_list:
            # Check if 'person' exists in dict AND follows 'user' back
            if person in friends and user in friends[person]:
                count += 1

        result[user] = count

    return result

print(reciprocal_follow_count(friends))
```

Python Q2: Friend of a Friend Recommendations

Question:

Given the same friends dictionary, write a function to return friend recommendations for each user. A user should be recommended if they are a friend of a friend but not already a direct friend or the user themselves.

Sample Input:

```
friends = {
    'a': ['b', 'c', 'd'],
    'b': ['a', 'c', 'd', 'e'],
    'c': ['a', 'e', 'f'],
    'd': ['a', 'e', 'f'],
    'e': ['b', 'c', 'f'],
    'f': ['c', 'd', 'f']
}
```

Expected output:

```
{
    'a': ['e', 'f'],
    'b': ['f'],
    'c': ['b', 'd'],
    'd': ['b', 'c'],
    'e': ['a', 'd'],
    'f': ['a', 'b']
}
```

1. Loop through each user
 2. Get their direct friends list
 3. For each direct friend, look at THEIR friends (friends of friends)
 4. Add those to a "recommended" set
 5. Remove: already direct friends + the user themselves
 6. What's left = recommendations!
- i

```
def friend_recommendations(friends):
    result = {}

    for user, direct_friends in friends.items():

        recommended = set()

        for friend in direct_friends:
            # .get() returns empty list if friend not found - no error!
            for fof in friends.get(friend, []):

                if fof != user and \
                    fof not in direct_friends:
                    recommended.add(fof)

        result[user] = sorted(recommended)

    return result

print(friend_recommendations(friends))
```

Combined Q1 & Q2 (Version 1&2 Combined)

You are given a `user_follows` dictionary with users and people they follow. Generate an output that states each and every user, and their reciprocal count and follow recommendations in a tuple.

Input:

```
user_follows = {
    'A': ['B', 'C', 'D'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'E'],
    'D': [],
    'E': ['F'],
    'F': ['E']
}
```

```
user_follows = {
    'A': ['B', 'C', 'D'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'E'],
    'D': [],
    'E': ['F'],
    'F': ['E']
}
```

```
'A': (2, ['E']),      # reciprocal: B, C | recs: E
'B': (2, ['E']),      # reciprocal: A, C | recs: E
'C': (2, ['D', 'F']), # reciprocal: A, B | recs: D, F
'D': (0, []),         # reciprocal: none | recs: none
'E': (1, []),         # reciprocal: F | recs: none
'F': (1, []),         # reciprocal: E | recs: none
```

```
def combined(user_follows):
    result = {}

    for user, direct_friends in user_follows.items():

        # ---- Q1: Reciprocal Count ----
        count = 0
        for person in direct_friends:
            if user in user_follows.get(person, []):
                count += 1

        # ---- Q2: Friend Recommendations ----
        recommended = set()
        for friend in direct_friends:
            for fof in user_follows.get(friend, []):
                if fof != user and \
                    fof not in direct_friends:
                    recommended.add(fof)

        # ---- Combine as tuple ----
        result[user] = (count, sorted(recommended))

    return result

output = combined(user_follows)
for user, value in output.items():
    print(f"{user}: {value}")
```

Set - 6

Car Rental Service

Product Sense

Clarifying Questions -

1. Company Context: "Are we an established player like Enterprise/Hertz, or a newer entrant? What's our current footprint?"

2. Goals: "What's the primary objective - maximize revenue, increase market share, or serve underserved markets?"

3. Constraints: "Do we have budget limitations? Any restrictions on fleet size or capital?"

4. Timeline: "Is this a near-term expansion (next 6-12 months) or longer-term strategic planning?"

Where should we expand our car rental locations?

Demand

- I would analyze our reservation and search data to understand where demand exists that we are not capturing
- High searches plus low conversion plus customers far from current locations = expansion opportunity

Supply

- If demand exist, but we can't efficiently manage fleet in a new market or if you're constantly stock of existing locations we need to fix operations before expanding
- Frequent stockouts in a specific vehicle categories plus high utilization plus one-way rental patterns pointing to a specific area

ROI

- We could have a high demand somewhere, but if economics don't work, let low ADR average daily rate high real estate cost heavy discounted needed to compete. It's a bad expansion, right
- Markets where we can command premium area have healthy. Add on attach and achieve sub 18 month payback or priority expansions.

Competition

- A market might look great on demand, but if it's saturated with five competitors and has excellent transit, our opportunity might be limited conversely, a market with weak public transit and growing business travel is perfect
- Low competitor density plus strong travel or activity drivers plus V transit alternatives

Geographical factors

- A location might work short-term, but if the Neighbourhood is declining, whether makes rentals seasonal or axis is terrible it's not sustainable
- Proximity to multiple demand generators plus stable or growing population plus your round demand plus easy axis equals to durable location

Customers

- I understanding who our customers are and where they're coming from helps identify loyalty, opportunities, and geographic gaps
- Neighborhoods with many repeat customers + high loyalty + customer traveling long distances to current locations = prime for expansion

A data show that certain rental car locations consistently ranked in the top five in terms of revenue. What metrics would you look at to understand contributions in their success?

Demand

- ~~Total~~ Demand reservations - total completed and booking in a period
- Search to reserve conversion - ratio of bookings to qualified searches
- ~~Total~~ reservations - total completed and booking in a period
- Search to reserve conversion - ratio of bookings to qualified search

Supply

Supply

- Fleet utilization rate - percentage of car days actually rented
- ~~Fleet~~ Fleet utilization rate - percentage of car days actually rented
- Maintenance turn around time - time between car return and rent ready
- ~~Fleet~~ Fleet to rental ratio - proportion of fleet that's rentable
- Fleet to rental ratio - proportion of fleet that's rentable
- Fleet occupancy rate - percentage of fleet booked at the moment
- Occupancy rate - percentage of fleet booked at the moment
- Average wait time - time between reservation and pick up
- Average wait time - time between reservation and pick up

Revenue

Revenue

- ADR (avg daily rate) - revenue per rental day
- ~~ADR~~ Add-on attach rate - percentage of reservations with extras insurance, etc.
- ~~Add-on~~ Attach rate - percentage of reservations with extras insurance, etc.
- Avg booking value - total revenue plus bookings
- Avg booking value - total revenue plus bookings
- Rental rate - revenue plus number of bookings

Customer Metrics

- Repeat rate - percentage of customers rebooking in timeframe, 30, 60, 90 days
- NPS - promoters minus detractors
- Customer Retention rate - percentage of customers active overtime
- Customer Satisfaction score - sustained demand and stabilize future revenue

Profitability

- Rental Revenue Per vehicle - average revenue for vehicle
- Dollar Utilization - annual revenue plus acquisition cost
- Net Profit Margin - revenue minus cost divided by revenue

We want to build a dashboard that shows the most critical information to the leadership team for our car rental business. What steps would you follow? Describe your end to end workflow in building such a dashboard

Clarifying questions

- Who is the audience?
- What decisions will they drive?
- What is cadence that we are expecting?

I would align build, and then launch

Requirement alignment

- Understand what decisions leadership needs to make like expansion, pricing, tweaks, or operational monitoring

Understanding audience

- Map users executives want high-level KPI analyst want drill downs to trailer the complex city accordingly

KPI framework and thresholds

- Picking 527 key metrics adding color-coded thresholds to the leaders glands what require attention

*Metric specification
define core KPIs with exact
formulas*

Data source mapping

- List all sources, reservations, fleet pricing, customer feedback, identify gaps of strength and plan how to collect missing data

Data modeling

- Build clean schemas with fact tables and dimension tables and a pre-compute daily aggregate for fast queries

ETL or ELT pipelines

- Design incremental pipelines with the retry logic and backfill capability

Data quality controls

- Implement checks like data, freshness, record counts business, logic, validation, and trigger alerts

Visualization and dashboard design

- Top summary KPI's middle trendiness bottom drill down filters keep it clean max seven metrics per screen, use maps and funnels where helpful

QA rollout and documentation

- Validate against source data, dry, and dry run with leadership document KPI definitions will use a guidance at axis control controls

Iteration and feedback loop

- Gather feedback and hydrate or update based on the new needs and data insights

Expansion opportunities the leadership team wants to identify expansion opportunities for additional locations Let's assume that you do not have any data how do you go about getting Data?

Scenario A - we have no internal data when entering completely new markets leverage external sources

Competitor Data

- Scrape price listings from Hertz, Avis, Enterprise
- Use for revenue potential

Search and demand signals

- We can get the data from the Google trends
- Use for validate real demand (car rental near me)

Government and public data

- US census bureau, AirPorts, DMB registrations
- Use for size the market population and income, airport passenger volume, vehicle ownership rates

Third party mobility data

- Purchase from third-party data providers
- Use travel patterns, demographic data

Partnership data

- Airlines, hotels, travel agencies
- Partner for the data or the referral programs

Scenario B - we have some limited data when we have internal data we can still extract value

Sampling and extrapolation

- Internal app logs (search, bookings)
- Extrapolate from sample to estimate total demand

Cohort analysis

- Historical data from similar cities or markets
- Identifying comparable data, applying its growth trajectory

Proxy metrics

- App downloads, website visits, location searches
- Interest signals when booking data sparse

Confidence intervals

- Statistical modeling of sparse data
- Booking estimated ranges providing upper and lower lower bounds

Scenario C - primary research when secondary data is insufficient

Surveys

- Target, digital ads email campaigns airport surveys
- Direct customer intent

Focus groups

- Recruit target customers locally to help give the pain points of the qualitative insights
- Understand why competitors is a succeed or fail

Pilot or soft launch

- Deploy Small fleet to understand real utilization data

Geo targeted A/B marketing

- Renards via Google ads or meta ads
- To understand, click through and sign up interest by region

Local partnerships

- Partner with the hotels, both corporate offices to understand, the travel demand and the referral volume

Data Modeling

Create a data model for the biometrics mentioned for a car rental agency

Key Entities

- Users
- Vehicles
- Locations
- Reservations

fct_reservation

Reservation ID
User ID
Vehicle ID
Reservation type
Reservation request TS
Reservation confirmed TS
Pick up TS
Return TS
Expected return TS
Pick up location ID
drop off location ID
pick up odometer
Drop off odometer
Total distance traveled
Payment method ID
Fare amount
Reservation status
Canceled by
Cancelled Ts
Reservation channel -(mobile/web)
Date (partition)

Dim user

User ID
Name
Gender
Age
Email
City
STATE
Country
Registration TS
Loyalty status
Last active TS
Date

Dim Vehicle

Vehicle ID
Make
Model
Year
Brand
License plate
Capacity
Vehicle type
Fuel type
Insurance expiry

Dim location

Relationship -

Dim - Fact : 1:M
Fact - Dim : 1:1

Location ID
Latitude
Longitude
City
State
Zip
Country
Location type

For each reservation if you want to store additional drivers related information, how would you do it?

Fct table

Additional drivers (list
or map)

Or

Bridge table (M:M)

fct_reserve_drivers

Reservation ID

User ID

Driver id

Driver name

Driver DL

SQL

Dim_cars

Car_id,
Make
Model
Year
Size
License_plate_number
Location id

Dim_locations

Location.id
address
City
State
Zip
Country
Type

Dim users

User is
Name
Email
License-number
License-state

Fct-rentals

Rental ID
User ID
Car ID
Pick up location ID
Drop off location ID
Pick up time
Drop off time
Rate per day

We want to know the matrices for rentals picked up in California last year by car size

Number of rentals

Number of unique users

Ratio of rentals per unique user

SELECT

c.Size AS car_size,
COUNT(r.Rental_ID) AS number_of_rentals,
COUNT(DISTINCT r.User_ID) AS number_of_unique_users,
ROUND(COUNT(r.Rental_ID) * 1.0 / COUNT(DISTINCT r.User_ID),
2) AS ratio_rentals_per_user

FROM Fct_rentals r

INNER JOIN Dim_cars c

ON r.Car_ID = c.Car_id

INNER JOIN Dim_locations l

ON r.Pick_up_location_ID = l.Location_id

WHERE l.State = 'CA'

AND YEAR(r.Pick_up_time) = YEAR(CURRENT_DATE) - 1

GROUP BY c.Size

ORDER BY number_of_rentals DESC;

We are doing inner join because only rentals that have valid car & location data.

We want to understand utilization per location and car size. We have defined a utilization metric based on availability versus rentals all rental pick ups and drop offs happen at the Car home location and cars are part of that home location inventory location, ID in cars.

The formula for utilization is as follows

Utilization rate = number of car rentals / number of cars in location

Write a SQL query to calculate utilization rate for each location and car sizes for today

```
WITH currently_rented AS (
    -- Count cars currently rented today by home location and size
    SELECT
        c.Location_id,
        c.Size,
        COUNT(DISTINCT r.Car_ID) AS cars_rented_today
    FROM Fct_rentals r
    INNER JOIN Dim_cars c ON r.Car_ID = c.Car_id
    WHERE CURRENT_DATE BETWEEN DATE(r.Pick_up_time) AND
DATE(r.Drop_off_time)
    GROUP BY c.Location_id, c.Size
),

total_inventory AS (
    -- Count total cars at each location by size
    SELECT
        Location_id,
        Size,
        COUNT(Car_id) AS total_cars
    FROM Dim_cars
    GROUP BY Location_id, Size
)

-- Calculate utilization rate
SELECT
    l.City,
    l.State,
    i.Location_id,
    i.Size,
    COALESCE(r.cars_rented_today, 0) AS cars_rented_today,
    i.total_cars,
    ROUND(COALESCE(r.cars_rented_today, 0) * 100.0 / i.total_cars,
2) AS utilization_rate_pct
FROM total_inventory i
LEFT JOIN currently_rented r
    ON i.Location_id = r.Location_id
    AND i.Size = r.Size
INNER JOIN Dim_locations l
    ON i.Location_id = l.Location_id
ORDER BY l.City, i.Size;
```

PYTHON

Python

Design and implement a Python function `manage_car_bookings(single_car_bookings_plan)` that simulates booking requests for a single car. The function should take a dictionary `single_car_bookings_plan` as input, where keys represent booking IDs and values are dictionaries containing `start_time` and `end_time` strings in "YYYY-MM-DD HH:MM:SS" format.

The function should process each booking request in the order of its booking ID (from 1 to N) and determine if it can be successfully accommodated. A booking is considered "ok" if it does not overlap with any previously accepted bookings. If a booking overlaps, it should be "rejected".

The function should print the status of each booking in the format "Booking [ID] [status]", where [status] is either "ok" or "rejected".

Example Input:

```
single_car_bookings_plan = {
    1: {"start_time": "2025-06-24 10:00:00",
        "end_time": "2025-06-24 11:00:00"},
    2: {"start_time": "2025-06-24 11:30:00",
        "end_time": "2025-06-24 12:30:00"},
    3: {"start_time": "2025-06-24 10:00:00",
        "end_time": "2025-06-24 10:30:00"},
    4: {"start_time": "2025-06-24 13:00:00",
        "end_time": "2025-06-24 14:00:00"},
    5: {"start_time": "2025-06-24 14:30:00",
        "end_time": "2025-06-24 15:30:00"},
    6: {"start_time": "2025-06-24 15:00:00",
        "end_time": "2025-06-24 16:00:00"},
    7: {"start_time": "2025-06-24 10:00:00",
        "end_time": "2025-06-24 11:15:00"},
    8: {"start_time": "2025-06-24 16:30:00",
        "end_time": "2025-06-24 17:30:00"}
}
```

Expected output

```
Booking 1 ok
Booking 2 ok
Booking 3 rejected
Booking 4 ok
Booking 5 ok
Booking 6 rejected
Booking 7 rejected.
Booking 8 ok
```

```
from datetime import datetime
```

```
def manage_car_bookings(single_car_bookings_plan):
```

```
    accepted_bookings = []
```

```
    for booking_id in sorted(single_car_bookings_plan.keys()):
```

```
        booking = single_car_bookings_plan[booking_id]
```

```
        start_time = datetime.strptime(booking["start_time"], "%Y-%m-%d %H:%M:%S")
```

```
        end_time = datetime.strptime(booking["end_time"], "%Y-%m-%d %H:%M:%S")
```

```
        is_overlap = False
```

```
        for accepted in accepted_bookings:
```

```
            accepted_start = accepted["start_time"]
```

```
            accepted_end = accepted["end_time"]
```

```
            if start_time < accepted_end and end_time > accepted_start:
```

```
                is_overlap = True
```

```
                break
```

```
        if is_overlap:
```

```
            print(f"Booking {booking_id} rejected")
```

```
        else:
```

```
            print(f"Booking {booking_id} ok")
```

```
            accepted_bookings.append({
```

```
                "start_time": start_time,
```

```
                "end_time": end_time
```

```
            })
```

`cars = [7, 8, 9, 10, 11]`

ordered dict of rental orders

assign available car to each order or reject
if no car is available for requested dates
car is assigned, print which car?

`available cars = [7, 8, 9, 10, 11]`

`rental_orders_by_id = {`

`21: {'pickup-date': , 'drop-off date': },`

`}`

`def process_orders (available_cars, rental_orders
by_id):`

`car_bookings = { }`

`for car_id in available_cars:`

`car_bookings[car_id] = { }`

`for order_id in rental_orders_by_id.keys():`

`pickup_date = rental_orders_by_id['pickup
date']`

`order_date = ['order_date']`

`assigned_car = False`

`for car_id in available_cars:`

bookings = car-bookings [car id]

availability = True ~~if~~ assume car is available

for existing pic, exist dup in booking:

if ^{not} (pickup > existing dropoff or
existing pickup > dropoff):

availability = false
break

if availability:

assigned car = car id
break

if assigned-car is not None:

car-bookings [booking-id] append

((pickup, dropoff))

Print (order { booking-id } ok -

assigned car { assigned-car })

else

print (Order { } rejected -
no car available)

set 7

Product Sense

Product Scenario

You're a data engineer supporting a messaging app with chat, 1:1 audio/video calling. The team launched a group video calling feature (up to 32 participants) with dynamic video quality adjustment, bandwidth optimization, and seamless participant management. The Product team wants to understand its performance and impact on overall app engagement.

- Is the group video calling feature available to all users or a specific subset (e.g., premium users only)?
- Is it available across all platforms (iOS, Android, Web)?
- When exactly was the feature launched? How long has it been live?

Question 1: Business Questions to Understand Performance

Engagement

1. Has overall app usage increased post-launch?
2. Are users engaging more with group calls vs 1:1 calls?
3. Is group calling cannibalizing 1:1 call usage?

Retention

1. Are group call users retained better than non-users?
2. Has overall retention improved post-launch?

Satisfaction

1. Are users satisfied with the group call experience?
2. Has overall app sentiment improved post-launch?
3. Have support issues spiked since launch?

Revenue

1. How much revenue is the group call feature generating?
2. Is it outperforming other features in revenue?
3. Has it contributed to overall revenue growth?
4. Are group call users upgrading to premium plans?

Acquisition

1. Is the feature attracting new users to the platform?
2. Is it a key driver for new sign-ups?

Referral

1. Have user invitations/referrals increased post-launch?
2. Are group call users inviting more people than 1:1 users?

Call Quality & Technical Performance

1. What is the avg connection success rate for group calls?
2. How often do users drop or rejoin a call?

Participant Behavior

1. What is the avg number of participants per group call?
2. What is the avg participation duration per user?
3. What % of calls are by repeat vs first-time initiators?

Feature Adoption & Usage

1. What time of day/week sees highest group call activity?
2. Do users who try group calling once become repeat users?

Network & Device Impact

1. Which device type has highest connection success rates?
2. How does call quality differ between cellular vs WiFi users?

Business Impact

1. Do group call users have higher app engagement than non-users?
2. Are group call users more likely to convert to premium?

Question 2: Key Metrics

Engagement Metrics

1. DAU/WAU/MAU

- Definition: Unique users active daily/weekly/monthly
- Formula: Count of unique user_id in a given time period

2. Avg time spent per user

- Definition: Average time spent on app per session
- Formula: Total time spent / Total number of users

3. Number of calls

- Definition: Total calls made in a given period
- Formula: Count of all call_id in a given time period

4. % share of group calls

- Definition: Group calls as a % of total calls
- Formula: (Group calls / Total calls) × 100

5. Total & avg call duration

- Definition: Total/average length of calls in minutes
- Formula: Total call duration / Total number of calls

6. % change in call frequency

- Definition: How call frequency shifted between features
- Formula: ((Post launch calls - Pre launch calls) / Pre launch calls) × 100

Retention Metrics

1. Retention rate

- Definition: % of users who return after a given period
- Formula: $(\text{Users at end of period} / \text{Users at start of period}) \times 100$

2. Churn rate

- Definition: % of users who stopped using the app
- Formula: $(\text{Users lost in period} / \text{Users at start of period}) \times 100$

3. Stickiness ratio

- Definition: How frequently users return to the app
- Formula: $\text{DAU} / \text{MAU} \times 100$

4. Repeat usage rate

- Definition: % of users using group calling more than once
- Formula: $(\text{Users who used group call 2+ times} / \text{Total group call users}) \times 100$

5. % change in retention rate

- Definition: Retention rate pre vs post launch
- Formula: $((\text{Post launch retention} - \text{Pre launch retention}) / \text{Pre launch retention}) \times 100$

Satisfaction Metrics

1. CSAT score

- Definition: User satisfaction rating after using feature
- Formula: $(\text{Satisfied users} / \text{Total survey respondents}) \times 100$

2. NPS score

- Definition: Likelihood of users recommending the app
- Formula: $\% \text{ Promoters} - \% \text{ Detractors}$

3. Avg app store rating

- Definition: Overall rating on iOS/Android stores
- Formula: $\text{Sum of all ratings} / \text{Total number of ratings}$

4. Sentiment score

- Definition: Positive vs negative tone from reviews
- Formula: $(\text{Positive reviews} / \text{Total reviews}) \times 100$

5. # of new support issues

- Definition: New bugs/complaints raised post launch
- Formula: Count of new support tickets post launch

Revenue Metrics

1. Subscription revenue

- Definition: Revenue tied to group video usage plans
- Formula: Number of subscribers × Subscription price

2. In-app purchase revenue

- Definition: Revenue from purchases related to feature
- Formula: Sum of all in-app purchases related to group calls

3. ARPU

- Definition: Average revenue generated per user
- Formula: Total revenue / Total number of active users

4. Revenue share by feature

- Definition: % of total revenue from group calls
- Formula: (Group call revenue / Total app revenue) × 100

5. Total revenue growth rate

- Definition: Overall revenue growth post launch
- Formula: ((Post launch revenue - Pre launch revenue) / Pre launch revenue) × 100

Acquisition Metrics

1. # of new user signups

- Definition: Total new users joining post launch
- Formula: Count of new user_id post launch

2. % of new users using group calls

- Definition: How many new users tried the feature
- Formula: (New users who used group calls / Total new users) × 100

Acquisition Metrics

Referral Metrics

1. Number of invites sent per user

- Definition: Avg invites sent by each user
- Formula: Total invi. tes sent / Total number of users

2. Referral conversion rate

- Definition: % of invited users who actually sign up
- Formula: (Users who signed up via invite / Total invites sent) × 100

Question 3: Dimensions

User

1. User Type - Free vs Premium user
2. Interests - User's areas of interest
3. Age - Age group of the user
4. Gender - Gender of the user
5. Signup Date - When the user joined the platform
6. Preferred Language - User's preferred app language
7. Origin Country - Country where user signed up from

User Behavior

1. New vs Returning - Whether user is new or existing
2. Frequency of Use - How often user uses the app
3. Viewing Patterns - When and how user engages with the app

Time

1. Day of Week - Which day the activity occurred
2. Weekend vs Weekday - Whether activity was on weekend or weekday
3. Seasonality - Trends based on seasons or holidays

Device

1. Make - Brand of device (Apple, Samsung etc.)
2. Model - Specific device model
3. App Version - Version of the app being used
4. Browser Type - Browser used if on web

Location

1. Country - Country of the user
2. Region - Region within the country
3. City - City of the user
4. Zip - Zip code of the user

Main Situation-Specific Dimensions

1. Feature Type

- Definition: Differentiates between 1:1 and group video calling
- Values: 1:1 video call vs Group video call

2. Time

- Definition: Compares performance before and after launch
- Values: Pre launch vs Post launch

3. Call Quality

- Definition: Quality and performance of the call
- Values: SD, HD / Bandwidth: High, Med, Low /

Participants: 1-5 low, 5-15 med, 16+ high

4. User Type

- Definition: Role of the user in the call
- Values: Admin/Call Initiator vs Participant/Joinee

Data Modeling

Data Modeling: Group Video-Call Tracking

Question / Problem Statement

Design a data model to track:

- How successful are group video-call connections and what affects call quality?
- How does group size impact call duration and participant engagement?
- Call Success Rate: Successful calls / Total call attempts
- Average Call Quality Score: Based on video resolution, latency, and drop rates
- Participant Retention: How long users stay in group calls

Approach: Identifying the Grain

Step 1: Understand the Grain

All metrics revolve around "calls" but exist at multiple grains:

- Group Call Level
- Participant Connection Level

Step 2: Map Metrics to Grain

Group Call Level

- What affects call quality?
- Group size & call duration
- Call Success Rate

Participant Connection Level

- How successful are individual connections?
- Participant Engagement (how long users stay)
- Average Call Quality Score
- Per-user metrics (resolution, latency, drop rate)

Step 3: Solution

Create 2 Fact Tables at 2 different granularities:

Fact Table 1 → fct_group_call_stats

- Grain: One row per group call session
- Captures: duration, participant count, call status

Fact Table 2 → fct_connections

- Grain: One row per user connection attempt
- Captures: quality metrics, join/leave times, connection status

Clarifying Questions -

Are we tracking calls at the overall session level or at the individual user level or both?

Call Success Rate - Was call successful?
(Lives at call)

Group size & Duration - How many people were
on call? How long was it?
(Lives at call)

Participation Retention - How long did each
person stay (different people stays for
different amount of time)
(Lives at user/connection level)

Avg call quality score - what was
the video quality?

Each person has different network condi
(Lives at user/connection level)

I can see these metrics live at two different levels - the call level and the connection level. So I'll need two fact tables."

fct_group_call_stats

- call_id (PK)
- call_admin_id
- start_ts
- end_ts
- call_duration_sec
- total_participant_cnt
- call_status (success/fail)
- date (partition)

dim_user

- user_id (PK)
- type (old/new)
- age_group
- gender
- signup_date
- date (snapshot partition)

dim_device

- device_id (PK)
- make
- model
- version
- date (snapshot partition)

dim_location

- location_id (PK)
- country
- region
- city

fct_connections

- connection_id (PK)
- call_id (FK)
- user_id (FK)
- device_id (FK)
- location_id (FK)
- start_ts
- end_ts
- connection_duration_sec
- avg_video_resolution
- avg_latency_rate
- avg_drop_rate
- avg_call_quality_score
- connection_status (success/fail)
- date (partition)

Call success rate

successful calls / total calls
FROM fct_group_call_stats

Avg call quality score

AVG(avg_call_quality_score)
FROM fct_connections

Participant Retention

SUM(connection_duration_sec)
GROUP BY user_id, call_id
FROM fct_connections

Follow-up: Handling a User Leaving and Rejoining Multiple Times

What if a user leaves and rejoins?"

Your answer:

"Because fct_connections is at the connection level, each join attempt gets its own row with a unique connection_id. So rejoins are handled naturally – we just sum up connection_duration_sec to get total time spent..

- A unique connection id is generated for every join attempt
- Each connection has its own start_ts and end_ts

Relationships -

fct_group_call_stats (1) —→ fct_connections (many)
One call can have many users connecting

dim_user (1) —→ fct_connections (many)
One user can join many different calls

dim_device (1) —→ fct_connections (many)
One device can be used across many calls

dim_location (1) —→ fct_connections (many)
Many users can connect from same location

dim_user (1) —→ fct_group_call_stats (many)
One user can have many calls

facts —> dims (1:1)

The Golden Rule

Each fact row can only have ONE value per foreign key
But that same dim record can appear in MANY fact rows

SQL

SQL - Group Video Calling Analysis

TABLE 1: fct_group_calls
Tracks high-level session metadata for every group video call.

call_id
start_time
end_time
initiator_user_id
max_participants
success_status
ds

Sample Data:

call_id	start_time	end_time	initiator_user_id	max_participants	success_status	ds
gc_001	2024-01-15 14:30:00	2024-01-15 15:00:00	1001	5	successful	2024-01-15
gc_002	2024-01-15 15:15:00	2024-01-15 15:18:00	1002	3	failed	2024-01-15
gc_003	2024-01-15 16:00:00	2024-01-15 16:45:00	1003	8	successful	2024-01-15

TABLE 2: fct_call_participants
Captures granular performance and connection metrics for individual participants.

call_id
user_id
join_time
leave_time
avg_quality_score
network_type
connection_success
ds

Sample Data:

call_id	user_id	join_time	leave_time	avg_quality_score	network_type	connection_success	ds
gc_001	1001	2024-01-15 14:30:00	2024-01-15 15:00:00	4.2	wifi	true	2024-01-15
gc_001	1004	2024-01-15 14:31:00	2024-01-15 15:00:00	3.8	cellular	true	2024-01-15
gc_002	1002	2024-01-15 15:15:00	2024-01-15 15:18:00	1.5	cellular	false	2024-01-15
gc_003	1003	2024-01-15 16:00:00	2024-01-15 16:45:00	4.5	wifi	true	2024-01-15

TABLE 3: dim_group_calls

group_call_id
call_start_time
call_end_time
initiator_user_id
max_participants
call_duration_seconds
call_quality_score
success_status

Sample Data:

group_call_id	call_start_time	call_end_time	initiator_user_id	max_participants	call_duration_seconds	call_quality_score
gc_001	2024-01-15 14:30:00	2024-01-15 15:00:00	1001	5	1800	10
gc_002	2024-01-15 15:15:00	2024-01-15 15:18:00	1002	3	180	0
gc_003	2024-01-15 16:00:00	2024-01-15 16:45:00	1003	8	2700	10

TABLE 4: fct_group_call_participants

```
group_call_id
call_join_id
user_id
device_id
join_time
leave_time
is_dropped
drop_reason
connection_success
network_type
country
```

Sample Data:

```
group_call_id | call_join_id | user_id | device_id | join_time | leave_time |
is_dropped | connection_success | network_type
gc_001 | cj_001 | 1001 | dev_101 | 2024-01-15 14:30:00 | 2024-01-15 15:00:00 | FALSE |
TRUE | wifi
gc_001 | cj_002 | 1004 | dev_104 | 2024-01-15 14:35:00 | 2024-01-15 15:00:00 | FALSE |
TRUE | cellular
gc_002 | cj_003 | 1002 | dev_102 | 2024-01-15 15:15:00 | 2024-01-15 15:18:00 | FALSE |
FALSE | cellular
gc_003 | cj_004 | 1003 | dev_103 | 2024-01-15 16:00:00 | 2024-01-15 16:45:00 | FALSE |
TRUE | wifi
```

QUESTION 1 (Schema 1)

Calculate connection success rate by network type (WiFi vs Cellular) for last 7 days. Only calls with 3+ participants.

filter - Last 7 days

(ds - 7)

- 3+ participants (max_participants > 3)

Group by - Network type (fct_call_partic)

Connection success rate =

$$\frac{\text{Count}(\text{connection_success})}{\text{Count}(*)}$$

Join both tables by call_id

```
WITH filtered_calls AS (
  SELECT call_id
  FROM fct_group_calls
  WHERE max_participants >= 3
  AND ds >= CURRENT_DATE - INTERVAL '7 days'
)
```

```
SELECT
  p.network_type,
  -- Numerator: successful connections (boolean true = 1)
  SUM(CASE WHEN p.connection_success = TRUE THEN 1 ELSE 0 END) * 1.0
  -- Denominator: all connection attempts
  / COUNT(*) AS connection_success_rate
FROM fct_call_participants p
WHERE p.ds >= CURRENT_DATE - INTERVAL '7 days'
INNER JOIN filtered_calls c
  ON p.call_id = c.call_id
GROUP BY p.network_type;
```

QUESTION 2 (Schema 1)

Top 10 initiators by total calling time in last 30 days. Include number of calls initiated.

extract(epoch from end_time - start_time) desc
initiator/user_id
limit 10
ds ≥ current_date() - 'Interval 30 days'
count(call_id) → # calls

```
SELECT
    initiator_user_id,
    COUNT(call_id) AS num_calls_initiated
FROM fct_group_calls
WHERE ds >= current_date - 30
GROUP BY initiator_user_id
ORDER BY SUM(EXTRACT(EPOCH FROM (end_time - start_time))) DESC
LIMIT 10;
```

QUESTION 3 (Schema 2)

Calculate the group call connection success rate by network type (WiFi vs Cellular) for the last 7 days. Include only calls with 3+ participants.

Connection success rate = successful participant connections / total connection attempts

Approach:

Connection success rate can be calculated with fct_group_call_participants, but calls with 3+ participants can be identified only from dim_group_calls.

max-participants ≥ 3 } dim_group_calls
group_call_id
call_start_time ≥ current_date - Interval 7 days

network_type
group_call_id
connection_success } fct_group_call_participants
join_time ≥ last 7 days

```

WITH filtered_calls AS (
    SELECT group_call_id
    FROM dim_group_calls
    WHERE max_participants >= 3
        AND DATE(call_start_time) >= CURRENT_DATE - INTERVAL '7 days'
)
SELECT
    p.network_type,
    SUM(CASE WHEN p.connection_success = TRUE THEN 1 ELSE 0 END) AS
successful_connections,
    COUNT(*) AS total_connection_attempts,
    ROUND(
        SUM(CASE WHEN p.connection_success = TRUE THEN 1 ELSE 0 END) * 1.0 /
COUNT(*),
        2
    ) AS connection_success_rate
FROM (SELECT * FROM fct_group_call_participants
    WHERE DATE(join_time) >= CURRENT_DATE - INTERVAL '7 days') p
INNER JOIN filtered_calls c
    ON p.group_call_id = c.group_call_id
WHERE p.network_type IS NOT NULL AND p.network_type != ''
GROUP BY p.network_type;

```

QUESTION 4 (Schema 2)

Find the top 10 group call initiators by total calling time in the last 30 days.
Include their total number of calls initiated and average call quality score.

Approach:

This schema has `call_duration_seconds` pre-calculated in `dim_group_calls`, making the query simpler – no need to calculate duration from timestamps.

dim_group_calls { initiator_user_id
call_start_time last 30 days
order by call_duration_seconds desc
limit 10
group by initiator_user_id
avg(call_quality_score) } metrics
Count(group_call_id)

```

SELECT
    initiator_user_id,
    COUNT(group_call_id) AS num_calls_initiated,
    ROUND(AVG(call_quality_score), 2) AS avg_call_quality_score
FROM dim_group_calls
WHERE call_start_time >= CURRENT_DATE - INTERVAL '30 days'
GROUP BY initiator_user_id
ORDER BY SUM(call_duration_seconds) DESC
LIMIT 10;

```

PYTHON

Python - Call Quality Insights (Group Video Calling)

Problem Context

The group video calling system processes real-time bandwidth and quality events to optimize call experience. The system receives streams of participant quality events and needs to make real-time decisions about video quality adjustments and participant management.

Input Data Structure

Each bandwidth event contains:

- `call_id` → string → Unique identifier for the group call
- `user_id` → int → Participant identifier
- `timestamp` → int → Unix timestamp when measurement was taken
- `bandwidth_up` → int → Upload bandwidth in kbps
- `bandwidth_down` → int → Download bandwidth in kbps
- `video_resolution` → int → Current video resolution (1=240p, 2=360p, 3=720p, 4=1080p)
- `participant_count` → int → Number of active participants in the call

Expected Input Format (from interview)

```
bandwidth_events = [
    {
        'call_id': 'gc_001',
        'user_id': 1001,
        'timestamp': 1640995200,
        'bandwidth_up': 700,
        'bandwidth_down': 1000,
        'video_resolution': 3,
        'participant_count': 4
    },
    {
        'call_id': 'gc_001',
        'user_id': 1001,
        'timestamp': 1640995260,
        'bandwidth_up': 400,
        'bandwidth_down': 500,
        'video_resolution': 2,
        'participant_count': 5
    },
    {
        'call_id': 'gc_001',
        'user_id': 1002,
        'timestamp': 1640995200,
        'bandwidth_up': 950,
        'bandwidth_down': 1500,
        'video_resolution': 4,
        'participant_count': 4
    }
]
```

Q1 Metrics (continued)

The function should calculate per call:

1. Average Total Bandwidth - Calculate average total bandwidth (`bandwidth_up` + `bandwidth_down`)
2. Recommended Video Resolution - Minimum `video_resolution` across all events in the call (conservative approach)
3. Low Bandwidth Users - Users who experience low total bandwidth (`bandwidth_up` + `bandwidth_down` < 1000) at least once during the call

Calculation Breakdown

- Event 1 (user 1001): 700 + 1000 = 1700 kbps (resolution: 3)
- Event 2 (user 1001): 400 + 500 = 900 kbps - Low bandwidth! (resolution: 2)
- Event 3 (user 1002): 950 + 1500 = 2450 kbps (resolution: 4)
- `avg_bandwidth_total`: (1700 + 900 + 2450) / 3 = 1683.33 (actual calculation)
- `recommended_video_resolution`: min(3, 2, 4) = 2
- `low_bandwidth_users`: [1001] (only user with bandwidth < 1000)

Note: The interview expected output shows 1717.0, but the correct calculation yields 1683.33. This appears to be a discrepancy in the original question. The solution produces the mathematically correct result.

Expected Output Format

```
{
  'gc_001': {
    'avg_bandwidth_total': 1717.0,
    'recommended_video_resolution': 2,
    'low_bandwidth_users': [1001],
  }
}
```

Solution Approach

1. Use defaultdict to automatically initialize state for each call_id
2. Iterate through every event in the input
3. For each event:
 - Accumulate total bandwidth for averaging
 - Track event count for averaging
 - Track minimum resolution (conservative quality recommendation)
 - Add user to low_bandwidth_users set if their bandwidth < 1000
4. Transform accumulated data into final output format

```
def process_bandwidth_events(bandwidth_events):

    call_data = {}

    for event in bandwidth_events:
        call_id = event['call_id']
        user_id = event['user_id']
        total_bw = event['bandwidth_up'] + event['bandwidth_down']
        resolution = event['video_resolution']

        # If call_id seen for first time, initialize it manually
        if call_id not in call_data:
            call_data[call_id] = {
                'total_bandwidth': 0,
                'event_count': 0,
                'min_resolution': float('inf'),
                'low_bandwidth_users': set()
            }

        # Now update the data
        call_data[call_id]['total_bandwidth'] += total_bw
        call_data[call_id]['event_count'] += 1
        call_data[call_id]['min_resolution'] = min(
            call_data[call_id]['min_resolution'], resolution)

        if total_bw < 1000:
            call_data[call_id]['low_bandwidth_users'].add(user_id)

    # Build final result
    result = {}
    for call_id, data in call_data.items():
        result[call_id] = {
            'avg_bandwidth_total': round(
                data['total_bandwidth'] / data['event_count'], 2),
            'recommended_video_resolution':
data['min_resolution'],
            'low_bandwidth_users': sorted(list(
                data['low_bandwidth_users']))
        }

    return result
```

Ownership

Introduction²
Focus Area
Autonomy
Leadership
Influence
Communications

Introduction

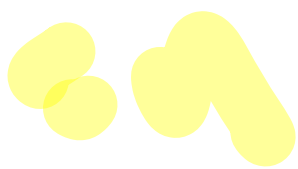
Hi I am [Name], a Data Engineer with around six years of experience across fintech, IoT, and enterprise data systems. I am currently at Samsara where I have spent the last two years doing everything from designing data architecture, building and maintaining pipelines, migrating thousands of data models across platforms, automating manual workflows with AI agents, managing data governance and access controls, ensuring data quality across critical business domains, to sitting with stakeholders and translating business problems into reliable data solutions. I work across Sales, Finance, HR, Product, and Customer Success – basically if a team at Samsara needs data to make a decision, it flows through infrastructure I have delivered.

Before Samsara I completed my Masters in Computer Science in the US specializing in NLP, which deepened how I think about data and intelligence systems.

Prior to that I spent two years at Forensic Alpha, an early stage fintech startup, owning nearly the entire data product end to end – ingestion, transformation, quality, and delivery of financial intelligence to institutional clients.

And I started my career at Freshworks as a Data and Machine Learning Engineer, where I got my first real taste of working with data at scale inside a fast moving product company.

What ties all of this together is simple – I am someone who is equally comfortable in the weeds of a broken pipeline at 2am and in a room full of stakeholders explaining why the data looks the way it does. That combination is what I bring to Meta.



Hi, my name is [Name] and I am currently a Data Engineer at Samsara where I have been for a little over two years now. But let me give you a bit of context on how I got here because I think it tells you a lot about who I am as an engineer.

I did my undergrad at SASTRA University in India where I actually scored first in both Machine Learning and Data Technologies. Not because I was the hardest working student in the room but honestly because I genuinely enjoyed it. I used to sit with datasets just exploring and lose track of time completely. That feeling told me something early on.

Right after graduating I joined Freshworks as a Data and Machine Learning Engineer. It was my first real exposure to working with data at scale inside a product company. And what I noticed about myself there was that I kept gravitating towards the foundation – the pipelines, the structure underneath everything – more than the modeling or the predictions on top.

That curiosity led me to Forensic Alpha, a very early stage startup where I was one of eight people and owned nearly 75% of the product. We were pulling financial data from annual reports, structuring it, and delivering it as intelligence reports to institutional clients. That experience taught me what real ownership feels like – if something broke I fixed it, if something needed to be built I built it. No safety net.

After two years there I made a decision that not everyone around me understood at the time – I left everything and came to the US to do my Masters, specifically to go deeper into this field. I specialized in NLP and Computer Science, landed an internship at Samsara as a Data Engineering intern, and that grew into a full time role where I have now been for over two years.

At Samsara I own three critical data domains simultaneously – the entire survey intelligence ecosystem that serves Customer Experience, HR, and Talent Acquisition, the most sensitive financial and people data in the company including compensation, headcount, and financial planning, and the product utilization intelligence that powers how Samsara understands device adoption across its entire customer base. A lot of this ownership came from stepping up when two senior engineers left the team within a short period with minimal handover. I absorbed both of their domains on top of my existing work and kept the team moving forward.

I am someone who does not just build pipelines. I build things the business cannot imagine operating without. And that is exactly the kind of engineer I want to be at Meta.

Focus Area

① How do you define the role of data engineer?

Analogy

- Raw simple teams
- Give example
- No technical
- strictly strict to analogy
- No tech jargon

ANALOGY

Film Editor

- Ending statement

I like to think of a data engineer as a **film editor**. When a movie is being made, the **director shoots** hundreds of hours of **raw footage** – multiple cameras, different angles, repeated takes, some brilliant and some completely unusable. The **editor's job** is to go **through all of that**, **cut** what does not belong, **clean up** what remains, **sequence everything** in the right order, and **stitch it together** into one coherent story that the **audience** can actually **follow and feel** something from. The actors do not think about the editing room. The audience never sees the cutting room floor. They just **watch a great film** and feel the emotion. But **without** the editor, all you have is hours of **disconnected footage** sitting on a hard drive that means absolutely nothing to anyone. That is **exactly what data looks like without a data engineer**.

Part 2 – Data Engineering Steps

In practice, this is how my work actually looks:

• **Requirement Gathering** – I start by sitting with business stakeholders to understand what decisions they are trying to make and what data they need to make those decisions.

• **Data Modeling** – I then design how that data should be structured and organized so it's easy to query, easy to understand, and ready to scale as the business grows.

• **ETL Development** – I build pipelines that Extract data from multiple source systems, Transform it into a clean and consistent format, and Load it into a central data warehouse or data lake.

• **Data Quality & Governance** – Throughout this, I put checks in place to make sure the data is accurate, complete, and trustworthy. I also make sure it's secure and access-controlled so the right people see the right data.

• **Analytics & Dashboards** – Finally, I set up dashboards, reports, and alerts so that analysts, managers, and executives can actually consume this data without needing to write a single line of code.

Part 3 - Ending Statement

At the end of the day, a data engineer is the person who makes sure that everyone in the organization – from a junior analyst to the CEO – is working from the same, reliable, high-quality data. We are essentially the backbone of the company's entire data ecosystem, and when we do our job right, good decisions just happen naturally.

Why data engineering ? Why did you choose this field ?

Honestly data engineering was never a plan. It was a series of moments that kept pulling me in the same direction.

It started at SASTRA University where I scored first in both Machine Learning and Data Technologies. But what stuck with me was not the grade – it was the feeling of sitting with a messy dataset and not being able to put it down until it made sense. That obsession told me something about myself early on.

When I joined Freshworks right after graduating I was officially a Machine Learning Engineer. But I kept finding myself more interested in where the data was coming from than what the model was doing with it. The pipelines, the structure, the foundation – that is where I naturally gravitated.

That instinct became impossible to ignore at Forensic Alpha. Eight people, no safety net, and I owned most of the product. We were taking raw financial data from annual reports – messy, inconsistent, scattered across hundreds of documents – and turning it into intelligence that institutional clients used to make real investment decisions. The first time I saw someone make a confident call based on data I had cleaned and structured, something clicked. That impact felt deeply personal.

That moment is honestly why I am in this field. Not because of a job description or a salary band. But because I discovered early that I am someone who finds genuine satisfaction in making messy things reliable. And once you find that feeling, you do not walk away from it.

How is data engineering different from data science or software engineering ?

Yeah, this is actually a question I get a lot and I love answering it because I've personally touched all three in some way.

So let me start with software engineering. A software engineer is essentially building the product – the application, the features, the user experience. Their world is around services, APIs, business logic. Data is not really their primary concern. They produce data as a byproduct of the product they build, but they're not responsible for what happens to that data afterwards.

Now data science sits on the other end. A data scientist is essentially the person asking – what does this data tell us? What can we predict? What patterns exist? They work with models, statistics, experiments. Their job is to extract insight and intelligence from data. But here's the thing – a data scientist is only as good as the data they receive. If the data coming in is messy, incomplete, or unreliable, the most sophisticated model in the world is going to give you a wrong answer.

And that's exactly where data engineering lives – right in the middle. We are the people who make sure that the data produced by software systems actually reaches the data scientists and analysts in a clean, structured, and trustworthy form. We build the pipelines, we design the warehouses, we handle the transformations, we set up the monitoring. We are essentially the plumbing that holds everything together.

I always say – software engineering builds the tap, data science drinks the water, and data engineering makes sure the water is clean by the time it gets there.

And having worked across all three in different capacities, I genuinely believe data engineering is the most underappreciated of the three – because when we do our job well, nobody notices. But the moment something breaks, everyone feels it.

What are the main things you do as a data engineer that strengthen your company

At Samsara I wear many hats as a data engineer but three things stand out as my biggest impact areas.

First is automation. I have eliminated hundreds of hours of manual work every month by automating data workflows that teams were doing by hand. One example – a monthly survey process that used to take multiple people several days now runs entirely on its own. Zero human intervention.

Second is data reliability at the executive level. I own pipelines that directly influence compensation for over 200 plus sales reps globally and financial planning decisions made by the CFO and HR leadership every single quarter. When those numbers are wrong the business feels it immediately. They have not been wrong on my watch.

Third is delivering what others could not. I inherited a product analytics project that had been sitting incomplete for three years and delivered it in under three months. That data layer now serves three major internal products simultaneously and is used daily by Customer Success, Sales, and Product teams across the company.

But beyond the projects – on a day to day basis I am doing everything from pipeline development, data modeling, migrations, data quality monitoring, access governance, stakeholder alignment, code reviews, and on call incident response. A data engineer at a high growth company is never just building models. You are the person the business calls when the data is wrong, when something breaks, and when nobody else knows where to look. That is the role I have owned at Samsara and that is what I bring to Meta.

AUTONOMY

There is always more work for a data engineer than you can possibly get done. How do you manage the situation?

Honestly this is something every data engineer faces but at Samsara it was particularly real for me.

The way I manage it comes down to three things I do consistently.

First I always separate urgency from importance. Not everything that feels urgent actually is. I sit with my manager, map every active project against real business impact and deadline sensitivity, and make that prioritization visible to everyone involved.

Second I phase everything. When I had five large projects running simultaneously I went back to every stakeholder and broke their work into phases. Phase one delivers the core value. Everything else follows. Nobody is blocked completely and I am always moving the most critical needle first.

Third I communicate proactively. The moment I know something is going to slip I tell the stakeholder before they come asking. That conversation is always easier early than late.

The specific example that shaped all of this – at one point I was simultaneously running the Xactly compensation rebuild, the Adaptive financial planning reimplementation, the product utilization delivery, and two other active projects while also on call. Five things. All urgent. All with senior stakeholders. I initially tried to context switch across all of them daily. Bad idea. Nothing was moving well. So I sat with my manager and we made a hard call – Xactly and Adaptive go first because hundreds of sales reps getting paid incorrectly and a CFO working with wrong headcount numbers are business critical. Everything else gets phased. Some stakeholders pushed back. I held the line. Everything got delivered. Just in the right order.

implementation
of bandwidth project
work together deliver
failing
When you
incorporated

What project that you worked on are you most proud of?

Situation:

At Samsara a critical project had been incomplete for nearly three years. Nobody had a single reliable view of device utilization across our entire customer base – how many devices purchased, returned, installed, activated, actively used, underutilized or overutilized. Data existed but lived across completely different systems that never agreed. Finance had one number from ACV contract values, NetSuite had another, Salesforce had a third. Account mergers, inconsistent returns, mismatched IDs – the same customer showing three different device counts depending on who you asked. Across thousands of enterprise customers and hundreds of thousands of physical IoT devices that is a serious business problem.

Task:

I inherited three years of ambiguity, misaligned stakeholders, and data sources that fundamentally disagreed with each other. My job was to build one number everyone could trust – from the ground up.

Action:

The first engineering decision I made was around data architecture. Rather than patching existing models I designed a clean layered architecture from scratch. I built a central fact orders table as the single foundation – pulling purchase quantities, return quantities, and net device counts at the customer level. On top of that I built a series of intermediate tables – fact license tables, device type level aggregations covering every applicable hardware SKU – VGs, CGs, cameras, all of it – combining applicable hardware and license data into unified device level metrics for the first time.

The hardest technical problem was resolving discrepancies at the source. Rather than hardcoding fixes or patching numbers at the model level I traced every discrepancy back to the upstream source and resolved it there. Account merger mismatches, inconsistent return logic, activation gaps – every issue was fixed at the root so the downstream models stayed clean and maintainable.

I also identified that R&D had built a widely used dim licenses table that the entire engineering org was pointing to. I replicated that functionality within our architecture, validated it was producing the same or more accurate results, and worked to migrate everyone off the old table and onto ours – making our models the new standard across the company.

Throughout all of this I ran structured UAT sessions with Finance, Customer Success, Sales, and Product simultaneously – comparing numbers across every system, adjusting filters based on team feedback, and validating that every metric was consistent across all consumers before anything went live.

Result:

Three year old problem closed in under three months. Over 20 data models delivered covering purchased, returned, net quantity, installed, activated, underutilized and overutilized metrics across hundreds of thousands of devices. Every team retired their own workaround and pointed to ours. Today the same utilization numbers power Admin Insights, Gainsight, AE Home, and executive reporting simultaneously. Within the first quarter Customer Success identified over 15% of enterprise accounts as underutilization risks enabling proactive outreach before renewals. One source of truth. Finally.

Tell me about the time you had to cut corners to meet at deadline what did you do?

Situation:

Samsara went through a company wide cost cutting initiative and the biggest target was our BigQuery spend. Leadership mandate – migrate everything off BigQuery onto Databricks. Our team owns all of Samsara's business data – Sales, Marketing, Finance, Support, Zendesk, everything. We were looking at somewhere between 1000 to 1500 data models. Three to four engineers including myself. Hard deadline. Non negotiable.

Task:

Nobody handed me a migration plan. I identified the scope, created the plan, and led the migration end to end. The biggest challenge was not the migration itself – it was figuring out what actually needed to move, what was legacy dead weight, and what would break something critical if it was missed.

Action:

I started with a full audit. I reverse engineered legacy systems, traced undocumented downstream dependencies, and mapped every model against active business usage. Anything untouched for over a year with no active stakeholder got flagged as low priority. That gave me a clear tiered picture – Tier 1 being business critical models powering live dashboards and executive reporting, Tier 2 being regularly used but lower visibility models, and Tier 3 being legacy models with unclear ownership. I structured the migration in two phases. Phase 1 covered all Tier 1 and Tier 2 models – roughly 80 to 85% of the total volume. Full migration, full syntax translation handling BigQuery to Databricks differences in date functions, partitioning logic, and raw source integrations, and full data parity validation between source and destination before anything went live. No shortcuts here because these models were powering real business decisions daily.

Phase 2 covered Tier 3 legacy models. And this is where we made a calculated tradeoff. Given the deadline and the shrinking team – engineers were leaving mid project – we migrated these models but skipped full regression testing. I flagged this explicitly to my manager, documented every model where testing was skipped, and we agreed on a post migration hotfix window specifically for these. We delivered approximately 90% of the full migration at production quality and accepted that the remaining 10% carried some risk that we had a plan to manage.

Midway through I was also absorbing work from engineers who left, carrying the majority of the migration load alongside my regular domain responsibilities. Context switching daily between migration work and keeping live pipelines running.

Result:

We hit the deadline. BigQuery shut down on time. Over 1000 data models successfully running on Databricks. A handful of Tier 3 models broke post launch exactly as anticipated – we resolved every single one within the hotfix window without any business disruption. What I am most proud of is not that everything was perfect. It is that I made a transparent, well reasoned engineering tradeoff – protected what mattered most, accepted calculated risk where the business could absorb it, and delivered on the commitment that actually counted. That is what senior engineering judgment looks like under real pressure.

OWNER SHIP

Tell me about a time you demonstrated initiative and took the lead on a project or tell me something that you thought yourself too or technology, language, topic, area, etc., and are proud of

Situation:

At Samsara we have a shared data channel where anyone in the company can come to our team with data questions, access requests, infrastructure support, Tableau issues – anything data related. One engineer owns that channel for an entire week on rotation. As the team grew and the number of projects expanded, the volume of incoming requests grew with it. We were getting anywhere between 20 to 30 requests per day. And what I noticed during my own on-call weeks was that I was not spending my time solving real data problems. I was spending it reading access tickets I did not fully understand, figuring out who to tag, waiting for manager approvals, and routing requests back and forth. Repetitive, manual, and completely unsustainable.

Task:

The moment that really hit me was watching one of my teammates during his on-call week. He was a strong engineer. But his entire week was consumed by access requests – maybe one or two real data problems got his attention all week. Everything else was just ticket routing. I thought – this person's skills are completely wasted on this. And so is everyone else's when it is their turn. I wanted to fix that.

Action:

I came up with the idea completely on my own, pitched it to my manager, and built it in my free time. I built an AI powered agent – a combination of a Slack bot and an LLM based agent with tool calling capabilities – that sits in the channel and handles incoming requests intelligently.

Here is what it actually does. When a request comes in the agent reads it, understands the intent, and categorizes it. If it is an access request – which was about 60 to 70% of all incoming tickets – the agent handles it end to end. It checks whether a relevant table or resource already exists in our silver layer and suggests it if it does. If new access needs to be provisioned it goes through our infrastructure as code setup – nothing is done manually, no AWS console, no manual permissions. Everything flows through proper approval channels in code. The agent automatically routes the request to the relevant manager for approval, tracks the response, and provisions access once approved. It also handles group creation if a new access group is needed. Essentially everything a support engineer was doing manually the agent now does automatically.

Result:

The team adopted it immediately. What used to consume the majority of an on-call engineer's entire week is now handled automatically. I would estimate the agent has effectively taken over the equivalent of two engineers worth of repetitive on-call work. The on-call engineer now spends their week focusing on actual data problems – pipeline issues, stakeholder questions that require real thinking, infrastructure improvements. The signal to noise ratio of the entire on-call experience changed completely. And honestly what I am most proud of is not the technology itself. It is that I saw a problem that everyone had accepted as just the way things are, I thought it made no sense, and I fixed it on my own time because I genuinely believed the team deserved better than spending their weeks routing access tickets. That is the kind of initiative I think separates someone who does the job from someone who improves how the job gets done.

If I were to join Facebook, what would your 90 day plan look like?

Honestly if I were to join Meta I would not walk in trying to prove myself on day one. I have learned from experience that the engineers who make the biggest long term impact are the ones who take the first few weeks to genuinely understand the environment before they start changing things.

Days 1 to 30 – Learn fast and find the quick wins:

My first priority would be getting up to speed as quickly as possible – not just the tech stack and internal tools but understanding how data flows through Meta's systems, who owns what, and where the pain points are. I would make the most of every onboarding session, dive deep into documentation, and set up one on ones with my manager, immediate teammates, and key stakeholders early. Not just to introduce myself but to genuinely listen – what is working, what is frustrating, what has been on the backlog for too long. I have found that in those early conversations people tell you exactly where the problems are. You just have to ask the right questions and actually listen. And within that first month I would be actively looking for a quick win – something small enough to deliver fast but meaningful enough to show the team how I think and work.

Days 30 to 60 – Start contributing and bring my strengths in: By this point I would have enough context to start contributing meaningfully. I would take ownership of a small to medium project and deliver it properly – not just to show I can execute but to start building the trust that earns you bigger responsibilities later. The strength I most want to bring to Meta early is AI and automation in data engineering. At Samsara I built agents that automated on-call access management and onboarding provisioning – saving the equivalent of two engineers worth of repetitive work. I genuinely believe there are opportunities at Meta's scale to apply that same thinking in ways that could have an outsized impact. I would be actively looking for those opportunities from day one and by day 60 I would want to have a concrete proposal on the table.

Days 60 to 90 – Build something nobody has built yet: By the third month I want to have made a mark that is uniquely mine. Not just delivered a project from the roadmap – but identified a problem or an opportunity that nobody had fully tackled yet and built something new around it. That is what genuinely excites me about Meta – the scale and the AI first culture combined. At Samsara I have been building AI agents to solve data engineering problems on a relatively small team. The idea of bringing that same thinking to Meta's scale – where the data volume, the infrastructure complexity, and the impact of getting it right is exponentially larger – that genuinely excites me. I want to see what I am capable of when the problems are that big. By day 90 I would sit down with my manager, share what I have learned, what I have built, and have an honest conversation about where I can make the most difference going forward.

Throughout all of this I would be seeking feedback constantly – not waiting for a review cycle but asking my manager and teammates regularly how I am doing and where I can improve. My goal in 90 days is simple. I want the team to feel that bringing me in was one of the best decisions they made this year.

How would you stay current in the general field of data engineering?

Honestly staying current in data engineering feels natural to me because I genuinely enjoy the field. It never feels like extra work.

From outside the company I stay connected through a few different channels. I subscribe to newsletters and blogs from the ecosystem – Databricks, dbt, Airflow, and engineering blogs from companies like Meta, Uber, and Airbnb because the problems they write about are real world and at scale. I follow practitioners on LinkedIn and Twitter who are actually building things and sharing learnings in real time. I attend conferences and webinars when I can and I am active in data engineering communities where people discuss new tools, patterns, and architectural shifts as they are happening – not after they are already mainstream.

From inside the company I learn just as much. I make it a point to have regular conversations with senior engineers and architects, participate in internal knowledge sharing sessions, and stay close to cross functional teams who are consuming the data I build. Those conversations surface problems and perspectives that no blog post ever would.

And then there is the AI angle which I think is now impossible to ignore. The way documentation is being generated, the way data pipelines are being built and monitored, the way access and governance are being automated – AI is changing what data engineering looks like from the inside out. I actively explore how LLMs and agents can improve or automate parts of the data engineering workflow. The agents I have already built at Samsara came directly from that curiosity and I think that is only going to become more important going forward.

Ultimately I stay current because I am genuinely curious about where this field is going and I want to be someone who is ahead of the change rather than catching up to it.

Can you tell me about a time when you felt disinterested in your work?

Honestly I think every engineer hits a point where the work starts feeling routine. I have felt it too and I think recognizing it early is actually a sign of self awareness rather than weakness.

For me when I notice that feeling creeping in the first thing I do is have a candid conversation with my manager. Not to complain but to be honest – I tell them I am looking for something more challenging, a new technology to explore, or a project that is completely different from what I have been doing. That conversation has always opened doors. My manager knows I am someone who thrives when I am slightly uncomfortable and learning something new so they have always found ways to give me that.

Outside of those conversations I also take matters into my own hands. The AI agents I built at Samsara – the onboarding agent and the on-call automation bot – both came from my own free time during a period where I wanted to explore something beyond my regular work. That personal initiative not only re-energized me but ended up creating real value for the team. So I have learned that when I feel disinterested the answer is usually to build something. Pick a problem, explore a new technology, and start experimenting.

I also find that mentoring junior engineers and participating in architecture discussions keeps me engaged. Teaching forces you to think deeper about things you think you already know and architecture conversations expose you to problems and perspectives outside your immediate domain.

And honestly right now with AI moving as fast as it is I cannot imagine feeling disinterested for long. There is so much to learn, so much to implement, and so much opportunity to improve how data engineering is done. Having that as a constant horizon keeps me genuinely motivated every day.

INFLUENCE

Tell me about a time you used data to influence a decision ?

OR

Describe a situation where you need to persuade someone to see things your way

Situation:

I built and automated our NPS survey targeting pipeline at Samsara – we survey roughly 3000 customer accounts every month. A stakeholder came to me flagging that 75% of this March's accounts were not matching last year's March data. That is a serious concern at that scale.

Task:

I needed to figure out whether my pipeline was actually broken or whether something else was going on – and come back with a clear answer backed by data.

Action:

I started by checking my code end to end. Everything looked fine. So I pulled every account we had surveyed month by month across the entire previous year and started mapping where this year's March data was actually landing. That is when I found it – this year's March accounts were not missing. They were spread across February, March, and April together. Which told me the comparison granularity was wrong. Digging further I realized the original manual process ran quarterly – Q1 surveyed as one full block. But the stakeholder had divided that Q1 block across individual months when comparing year on year. She was comparing a monthly slice against a quarterly distribution. They were never going to match.

Result:

I documented the finding, visualized the distribution, and showed that Q1 to Q1 the data aligned perfectly. My manager saw it immediately – not a pipeline issue, a methodology misalignment hiding in the old manual process. Four hours of investigation completely explained a 75% mismatch.

contacts →
after affects

Tell me a time you encountered a rule or a policy in your organization that you thought made no sense

Situation:

When a new engineer joins our data team at Samsara, the very first thing they need is Databricks access. Everything we do – every pipeline, every data model, every day to day task – runs through Databricks. But the access provisioning process required director level approvals, multiple manual requests across completely different teams, and new joiners had no idea who to contact or what to ask for. From day one of joining to the day someone could actually open Databricks and do real work was anywhere between three weeks to a month. For a small team already stretched thin, losing a new engineer's first month to waiting on approvals made absolutely no sense to me.

Task:

I could have just accepted it as the way things work. Everyone before me had. But I felt strongly that this was a process problem disguised as a policy problem and that somebody needed to actually fix it rather than just complain about it.

Action:

I built an AI powered onboarding agent. The moment a new employee receives their offer the agent kicks off automatically. It knows every tool the person needs, every group they need to be added to, and every approval that needs to happen including director level Databricks approvals. It automatically creates requests in the relevant teams, routes approvals proactively, and walks the new employee through every single step without them having to figure anything out on their own. No more chasing people. No more not knowing who to ask. No more sitting idle for weeks wondering what comes next.

Result:

What used to take three to four weeks now completes in under a week. New engineers are productive from day one. And it is not just Databricks – every tool, every group, every access requirement gets handled through the same agent. Something the entire team had silently accepted as just the way things are got completely reimaged because I thought it made no sense and decided to do something about it. That is honestly how I think about broken processes – if something is slowing the team down and the fix is within your reach, you fix it.

Communication

Can we tell me about the time when you had a professional conflict?

OR

What is an opinion you have held where someone was able to change your mind

Situation:

At Samsara I am the sole owner of some of the most sensitive data in the company – compensation data, headcount data, hiring data from Greenhouse. Because of the sensitivity of these sources, access is tightly controlled. Legal had explicitly granted permissions to only a specific set of people within the organization. That boundary existed for a reason.

Task:

One of our support teams needed Greenhouse interview data to support the candidate experience survey work we were doing. It was a legitimate business request. But the way it was being asked for created a real governance problem that I was not willing to ignore.

Action:

The request got escalated and eventually my manager suggested the pragmatic solution – just give the entire support group access. The reasoning was straightforward from an operational standpoint. Creating a restricted access group for only two specific people was additional work, additional overhead, and the team was already stretched thin. I understood the logic completely. But I pushed back. Not in a confrontational way – I sat down with my manager and walked through exactly why I was uncomfortable with the blanket access approach. Legal had drawn a specific line around who could see this data. Greenhouse contains candidate personal information – interview feedback, rejection reasons, hiring decisions. Giving broad group access to sensitive candidate data without going through proper governance channels was not just an operational risk. It was a legal and ethical one. I made that case clearly and calmly.

My manager heard me out. And rather than just disagreeing and escalating further, I came with a solution already in hand. I took the initiative to create a properly scoped access group, added only the two people who had legitimate need and legal clearance, documented the access decision, and delivered the data through the right channel. Same outcome for the business. Zero governance risk.

Result:

The support team got what they needed. The two people who needed access had it. And the sensitive candidate data stayed protected within the boundaries Legal had defined. My manager appreciated that I did not just flag the problem – I solved it the right way. That moment reinforced something I feel strongly about as a data engineer – when you are the sole owner of sensitive data, you are also the last line of defense for the people whose data you are protecting. Speed and convenience can never be the reason to compromise that.

Survey Intelligence Ecosystem:

Let me tell you about one of the most complex and cross-functional things I own at Samsara – and I use the word own very deliberately. Every survey that leaves Samsara – to any person, internal or external – flows through data infrastructure I designed and built from scratch. But let me give you the full picture of what that actually means because it is much larger than it sounds.

On the customer facing side, I own the NPS survey pipeline, driver satisfaction surveys, and operational support surveys. These go out to Samsara's enterprise customers and drivers at scale – we are talking about monthly automated targeting and delivery across a large and constantly changing customer base. The targeting logic, the automation that generates and pushes files to Qualtrics via API, the connectors that bring responses back in, the gold layer models, and the KPI dashboards that Customer Experience leadership consumes – all of that is mine.

But then on the people and hiring side, it gets even more interesting. I identified early on that Samsara had a significant blind spot – there was no structured data infrastructure around the employee and candidate experience lifecycle. So I took initiative, partnered with HR leadership, Talent Acquisition, and the Greenhouse team, aligned all of them around a unified data vision, and then built it. Today that infrastructure covers – candidate rejection surveys that automatically trigger when an active candidate is rejected in Greenhouse, 90 day quality of hire surveys that go to hiring managers after a new employee joins to measure how good the hire actually was, job closing surveys sent to hiring managers the moment a role closes, and exit surveys for departing employees. Each of these touches a completely different stakeholder – recruiters, hiring managers, HR business partners, and People Analytics – and each has its own unique triggering logic, targeting rules, and downstream reporting requirements.

What I am most proud of here is not any single pipeline. It is the fact that I sat down with Customer Experience, HR, Talent Acquisition, and Recruiting – four completely different functions with four completely different needs and vocabularies – and I designed a single unified survey data architecture that serves all of them reliably and consistently. I was not handed a requirements document. I drove the conversations, I asked the right questions, I made the architectural decisions, and I delivered something that none of these teams had before. Today across all survey domains combined we are running over 30 plus data models, serving more than 6 business functions, and what used to be a heavily manual monthly process for multiple teams now runs automatically with zero human intervention every single month.